

2025/26

U.D.2. CREACIÓN DE INTERFACES WEB HOMOGÉNEAS.

2.5 Propiedades CSS – Parte 2.





ÍNDICE

1. Propiedades de las tablas	4
1.1 'caption-side'	4
1.2 'empty-cells'	4
1.3 'border-collapse'	5
1.4 'border-spacing'	5
1.5 'table-layout'	6
2. Propiedades de las listas en CSS	7
2.1 'list-style-type'	7
2.2 'list-style-image'	8
2.3 'list-style-position'	9
2.4 'list-style'	10
2.5 Personalizar listas con emoticonos	10
2.6 Usar una imagen con background-image	11
3. Estilos en formularios	12
3.1 Propiedad resize	12
3.2 Estilos en el input: pseudo-clase :focus	13
3.2.1 Eliminar el contorno predeterminado	13
3.2.2 Estilizar el fondo y el borde	13
3.2.3 Animaciones y transiciones	13
3.3 Estilos para inputs	14
3.3.1 Estilo básico	14
3.3.2 Bordos redondeados	14
3.3.3 Sombras	15
3.3.4 Hover y focus	16
3.3.5 Iconos en inputs	16
3.3.6 Animaciones en inputs	18
3.4 Estilos para inputs requeridos, asterisco en label	19
4. Cursores en CSS	22
5. Definición de propiedades personalizadas	24
5.1 Uso de propiedades personalizadas	24
6. Funciones matemáticas	26
6.1 calc()	27
6.1.1 CSS calc() para mejorar la tipografía fluida	28
6.2 min()	29
6.3 max()	29



6.4	round()	30
6.5	attr()	31
6.6	CSS clamp(): tipografía fluida o responsive	33
6.7	CSS clamp() junto con calc() para cálculos matemáticos y establecer límites	34
7.	CSS line-clamp(): acortar texto a un nº de líneas.	35
7.1	Cómo implementar line-clamp en CSS	35
7.2	Ejemplos prácticos de line-clamp	36
7.2.1	Ejemplo 1. Truncado de texto en cards o tarjetas de contenido	37
7.2.2	Ejemplo 2. Acortar textos de los encabezados o subtítulos	38
7.2.3	Ejemplo 3. Acortar textos de productos en una tienda online	38
8.	Transform	40
8.1	Tipos de transformaciones.	41
9.	Transition	46
10.	Iconografía	49
10.1	Font Awesome	49
10.1.1	Registro u online	49
10.1.2	Descarga o local	51
10.2	Google Icons	52
10.2.1	Registro u online	52
10.2.2	Descarga o local	54
10.3	Icomoon	55
10.3.1	Registro u online	55
10.3.2	Descarga o local	57
11.	Imágenes SVG en HTML y CSS	59
11.1	Cómo insertar imágenes SVG	59
11.1.1	Usando la etiqueta para un SVG	59
11.1.2	Usando SVG en línea (inline)	60
11.1.3	Usando la etiqueta <object> para un SVG	61
11.1.4	Usando la etiqueta <iframe> con SVG	61
11.1.5	Usando la propiedad background-image con SVG	62
11.2	Cambiar las propiedades de un SVG con CSS	64
11.2.1	Cambiar el color de una imagen SVG	64
11.2.2	Ajustar el tamaño de una imagen SVG	64
11.2.3	Cambiar la opacidad de una imagen SVG	65
11.2.4	Cambiar la posición de una imagen SVG	66
11.2.5	Animar una imagen SVG con CSS	66
12.	Filtros CSS para aplicar efectos en imágenes	67



12.1	Ejemplos de filtros en CSS.....	69
13.	Blend Modes CSS.....	76
13.1	Tipos de Blend Modes y sus efectos visuales	76
13.2	Implementación de Blend Modes en CSS	77
13.3	Diferencias entre Blend Modes y Filtros CSS.....	79
13.4	Ejemplos prácticos de Blend Modes	79
13.4.1	Efectos de texto creativos con overlay	79
13.4.2	Mezclar un color con la imagen de fondo con color-dodge.....	80
13.4.3	Otros efectos interesantes	82
14.	El modo oscuro y claro con CSS.....	83
14.1	¿Qué es prefers-color-scheme?.....	83
14.2	Cómo usar prefers-color-scheme	83
15.	Scroll Snap en CSS	86
15.1	Ejemplos prácticos.....	88
15.2	Secciones de contenido que ocupan el 100% del alto.....	88
15.3	Galerías de productos en contenedores horizontales.....	90
15.4	Galería vertical con texto descriptivo.....	90
16.	Scroll-behaviour CSS	92
16.1	Propiedad CSS scroll-behavior: sintaxis y valores	92
16.2	Diferencias entre scroll-behavior y scroll-snap	92
16.3	Ejemplos prácticos de scroll-behavior	93
16.3.1	Menú de navegación con enlaces a secciones con scroll suave	93
16.3.2	Botón para volver a la parte superior de la página con scroll suave	95
16.3.3	Aside de contenidos con scroll suave	97
17.	Preprocesadores CSS	101
17.1	Preprocesadores Less y Sass.....	101
18.	CSS Nesting Nativo (Anidamiento CSS Nativo)	104
18.1	Sintaxis relaxed.....	105
18.2	Sintaxis avanzada.....	106
18.3	Anidamiento sobre el padre	107
18.4	Anidamiento de reglas	108
19.	Anexo I - SaSS	109
19.1	Instalación de SaSS.....	109
19.2	Formatos.....	110
19.3	Cómo compilar nuestro código.....	112



1. Propiedades de las tablas

En las páginas web existen muchos elementos específicos a los que debemos dar forma para así conseguir la apariencia que más nos interese. Uno de estos elementos son las tablas.

Para controlar la presentación de las tablas tenemos las propiedades: `caption-side`, `table-layout`, `border-collapse`, `border-spacing`, y `empty-cells`.

Propiedad	Descripción	Valores
<code>caption-side</code>	Posición del título respecto la tabla	top bottom
<code>table-layout</code>	Formato de las celdas, filas y columnas	auto fixed
<code>border-collapse</code>	Selección del modelo de los bordes	collapse separate
<code>border-spacing</code>	Espaciado entre los bordes de celdas adyacentes	longitud
<code>empty-cells</code>	Visibilidad de los bordes de celdas sin contenido	show hide

1.1 'caption-side'

Esta propiedad sirve para indicar **dónde se pone el título** de la tabla. Puede tener los valores: `top`, `bottom`, `left` y `right`.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    caption-side: top | bottom | initial | inherit;  
}
```

1.2 'empty-cells'

Esta propiedad soluciona la carencia del XHTML que, al no dibujar las celdas que estaban vacías, obligaba a poner un espacio en blanco usando el carácter ** ** (de espacio duro).

Los valores que admite son:

- `show` que permite mostrar los bordes y fondos como en las celdas con contenido.
- `hide` que permite ocultar los bordes y fondos de las celdas vacías.
- `inherit` que permite heredar el valor de `empty-cells` que tenga su elemento padre.



La **sintaxis** es la siguiente:

```
etiqueta html {  
    empty-cells: show | hide | inherit;  
}
```

1.3 'border-collapse'

Permite establecer el modo en el que se dibujan los bordes de las tablas: **separate** (separados), **collapse** (juntos) e **inherit**. En el modo **separate**, cada celda está rodeada por su borde haciendo el efecto de un borde con una línea doble, mientras que, en el modo **collapse** las celdas contiguas comparten sus bordes.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    border-collapse: separate | collapse | inherit;  
}
```

1.4 'border-spacing'

Permite establecer la **separación entre celdas contiguas**. Para hacerlo se indica el valor del espaciado horizontal seguido del valor del espaciado vertical. Si se escribe un único valor, la separación horizontal y vertical serán iguales.

Esta propiedad sólo funciona cuando border-collapse es separate.

Los valores que admite son:

- **length** que especifica la distancia entre los bordes de celdas adyacentes en px, cm, etc. No admite números negativos.
- **initial** que pone la propiedad a su valor por defecto.
- **inherit** que permite heredar el valor de border-spacing que tenga su elemento padre.

La **sintaxis** es la siguiente:

```
etiqueta html {  
    border-spacing: length | initial | inherit;  
}
```



1.5 'table-layout'

Permite definir el modo en el que el navegador dibujará la tabla ya que puede hacerse de dos formas.

Los valores que admite son:

- *fixed*: dibuja la tabla basándose en las medidas establecidas en el código fuente. Con este valor se consigue que el sistema trabaje más rápido.
- *auto*: dibuja la tabla basándose en el contenido de sus celdas. Es el valor **por defecto**.

A continuación, se muestra un **ejemplo** de todas las propiedades CSS de tablas vistas en este apartado:

```
<!DOCTYPE html>
<html>
<head>
<style>
table, td, th {
  border: 1px solid black;
}

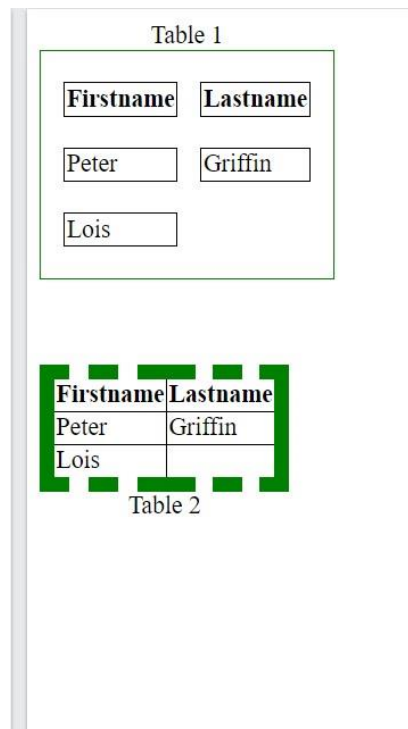
#table1 {
  border-collapse: separate;
  caption-side: top;
  empty-cells: hide;
  border-spacing: 15px 20px;
  border: 1px solid green;
  table-layout: auto;
}

#table2 {
  border-collapse: collapse;
  caption-side: bottom;
  empty-cells: show;
  table-layout: fixed;
  border: 10px dashed green;
}
</style>
</head>
<body>

<table id="table1">
<caption>Table 1 </caption>
<tr>
  <th>Firstname</th>
  <th>Lastname</th>
</tr>
<tr>
  <td>Peter</td>
  <td>Griffin</td>
</tr>
<tr>
  <td>Lois</td>
</tr>
</table>

<br><br><br>
<table id="table2">
<caption>Table 2 </caption>
<tr>
  <th>Firstname</th>
  <th>Lastname</th>
</tr>
<tr>
  <td>Peter</td>
  <td>Griffin</td>
</tr>
<tr>
  <td>Lois</td>
</tr>
</table>

</body>
</html>
```





2. Propiedades de las listas en CSS

Las propiedades CSS de las listas son las que nos permiten controlar los estilos de los marcadores y la posición de los elementos dentro de las listas. En la siguiente tabla se muestran **las propiedades de las listas** más utilizadas.

Propiedad	Descripción	Valores
<i>list-style-type</i>	Estilo aplicable a los marcadores visuales de las listas	disc circle square decimal decimal-leading-zero lower-roman upper-roman lower-greek lower-latin upper-latin armenian georgian lower-alpha upper-alpha none
<i>list-style-image</i>	Imagen aplicable a los elementos de las listas	url(«...») none
<i>list-style-position</i>	Posición del marcador dentro de la lista	inside outside
<i>list-style</i>	Permite establecer varios estilos de la lista en una sola propiedad	list-style-type list-style-position list-style-image

2.1 'list-style-type'

Esta propiedad CSS especifica el **tipo de marcador** de elemento en una lista.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    list-style-type: disc | circle | square | decimal | decimal-leading-zero | lower-roman | upper-roman | lower-greek | lower-latin | upper-latin | armenian | georgian | lower-alpha | upper-alpha | none;  
}
```

Se pueden ver los diferentes estilos en:

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_list-style-type

A continuación, se presenta un **ejemplo** sencillo:



```
<!DOCTYPE html>
<html>
<head>
<style>
.a {list-style-type: circle;}
.b {list-style-type: square;}
.c {list-style-type: upper-roman;}
.d {list-style-type: lower-alpha;}
</style>
</head>
<body>
<h3>Propiedad list-style-type</h3>
<ul class="a">
<li>Tipo círculo</li>
<li>Tipo círculo</li>
<li>Tipo círculo</li>
</ul>
<ul class="b">
<li>Tipo cuadrado</li>
<li>Tipo cuadrado</li>
<li>Tipo cuadrado</li>
</ul>
<p>Listas ordenadas:</p>
<ol class="c">
<li>Uno</li>
<li>Dos</li>
<li>Tres</li>
</ol>
<ol class="d">
<li>Uno</li>
<li>Dos</li>
<li>Tres</li>
</ol>
</body>
</html>
```

Propiedad list-style-type

- Tipo círculo
- Tipo círculo
- Tipo círculo
- Tipo cuadrado
- Tipo cuadrado
- Tipo cuadrado

Listas ordenadas:

- I. Uno
- II. Dos
- III. Tres
- a. Uno
- b. Dos
- c. Tres

2.2 'list-style-image'

Esta propiedad CSS **sustituye el marcador** de elemento de lista **por una imagen**.

Su **sintaxis** es la siguiente:

```
etiqueta html {
    list-style-image: none | url | initial | inherit;
}
```

A continuación, se presenta un ejemplo sencillo:

```
<!DOCTYPE html>
<html>
<head>
<style>
ul {
list-style-image: url('sqpurple.gif');
}
</style>
</head>
<body>

<h1>The list-style-image Property</h1>

<p>The list-style-image property replaces the
list-item marker with an image:</p>

<ul>
<li>Coffee</li>
<li>Tea</li>
<li>Coca Cola</li>
</ul>

</body>
</html>
```

The list-style-image Property

The list-style-image property replaces the list-item marker with an image:

- Coffee
- Tea
- Coca Cola



2.3 'list-style-position'

Esta propiedad especifica la posición de las viñetas o marcadores de los elementos de la lista. Su **sintaxis** es la siguiente:

```
etiqueta html {  
    list-style-position: inside | outside | initial | inherit;  
}
```

- **list-style-position: outside;** significa que las viñetas estarán fuera del elemento de la lista. El inicio de cada línea de un elemento de lista se alinearán verticalmente.

- Coffee - A brewed drink prepared from roasted coffee beans...
- Tea
- Coca-cola

- **list-style-position: inside;** significa que las viñetas estarán dentro del elemento de la lista. Al formar parte del elemento de la lista, formarán parte del texto y lo empujarán al principio

- Coffee - A brewed drink prepared from roasted coffee beans...
- Tea
- Coca-cola

Esta propiedad se puede ver en el siguiente **ejemplo**:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
ul.a {  
    list-style-position: outside;  
}  
  
ul.b {  
    list-style-position: inside;  
}  
</style>  
</head>  
<body>  
  
<h1>The list-style-position Property</h1>  
  
<h2>list-style-position: outside (default):</h2>  
<ul class="a">  
  <li>Coffee - A brewed drink prepared from roasted coffee beans, which are the seeds of berries from the Coffea plant</li>  
  <li>Tea - An aromatic beverage commonly prepared by pouring hot or boiling water over cured leaves of the Camellia sinensis, an evergreen shrub (bush) native to Asia</li>  
  <li>Coca Cola - A carbonated soft drink produced by The Coca-Cola Company. The drink's name refers to two of its original ingredients, which were kola nuts (a source of caffeine) and coca leaves</li>  
</ul>  
  
<h2>list-style-position: inside:</h2>  
<ul class="b">  
  <li>Coffee - A brewed drink prepared from roasted coffee beans, which are the seeds of berries from the Coffea plant</li>  
  <li>Tea - An aromatic beverage commonly prepared by pouring hot or boiling water over cured leaves of the Camellia sinensis, an evergreen shrub (bush) native to Asia</li>  
  <li>Coca Cola - A carbonated soft drink produced by The Coca-Cola Company. The drink's name refers to two of its original ingredients, which were kola nuts (a source of caffeine) and coca leaves</li>  
</ul>  
  
</body>  
</html>
```

The list-style-position Property

list-style-position: outside (default):

- Coffee - A brewed drink prepared from roasted coffee beans, which are the seeds of berries from the Coffea plant
- Tea - An aromatic beverage commonly prepared by pouring hot or boiling water over cured leaves of the Camellia sinensis, an evergreen shrub (bush) native to Asia
- Coca Cola - A carbonated soft drink produced by The Coca-Cola Company. The drink's name refers to two of its original ingredients, which were kola nuts (a source of caffeine) and coca leaves

list-style-position: inside:

- Coffee - A brewed drink prepared from roasted coffee beans, which are the seeds of berries from the Coffea plant
- Tea - An aromatic beverage commonly prepared by pouring hot or boiling water over cured leaves of the Camellia sinensis, an evergreen shrub (bush) native to Asia
- Coca Cola - A carbonated soft drink produced by The Coca-Cola Company. The drink's name refers to two of its original ingredients, which were kola nuts (a source of caffeine) and coca leaves



2.4 'list-style'

Esta propiedad CSS es una abreviatura de las propiedades anteriores, esto es, con esta propiedad podemos establecer **varios estilos de una lista en una sola línea** de código.

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    list-style: list-style-type list-style-position list-style-image  
               | initial | inherit;  
}
```

Ejemplo:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
ul {  
    list-style: square inside url("sqpurple.gif");  
}  
</style>  
</head>  
<body>  
  
<h1>The list-style Property</h1>  
  
<p>The list-style is a shorthand property for  
all the list properties.</p>  
  
<ul>  
    <li>Coffee</li>  
    <li>Tea</li>  
    <li>Coca Cola</li>  
</ul>  
  
</body>  
</html>
```

The list-style Property

The list-style is a shorthand property for all the list properties.

- Coffee
- Tea
- Coca Cola

2.5 Personalizar listas con emoticonos

Podemos personalizar nuestras listas utilizando emoticonos como marcadores. Para hacerlo, podemos utilizar la propiedad `list-style-image` en combinación con algunos emoticonos o iconos que deseemos utilizar. Además, necesitaremos usar el pseudo-elemento `::before`.

HTML

```

1 <ul>
2 <li>Elemento 1</li>
3 <li>Elemento 2</li>
4 <li>Elemento 3</li>
5 <li>Elemento 4</li>
6 <li>Elemento 5</li>
7 </ul>

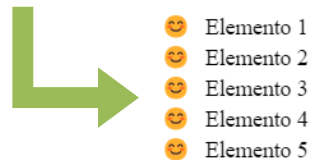
```

CSS

```

1 ul {
2     list-style: none; /* Eliminamos los marcadores predeterminados */
3     padding: 0; /* Eliminamos el relleno predeterminado */
4 }
5
6 li::before {
7     content: '😊'; /* Emoticono como marcador */
8     margin-right: 10px; /* Espacio entre el emoticono y el texto del elemento */
9 }

```



2.6 Usar una imagen con background-image

Ya hemos visto cómo agregar una imagen en la viñeta usando la propiedad `list-style-image`. Sin embargo, este método es muy limitado en lo que respecta al control del tamaño y posición de la viñeta. Para este caso, es más apropiado usar la propiedad `background-image`.

Supongamos que tenemos una imagen llamada “viñeta.png” que deseamos utilizar como marcador en nuestra lista:

HTML

```

1 <ul>
2 <li>Elemento 1</li>
3 <li>Elemento 2</li>
4 <li>Elemento 3</li>
5 <li>Elemento 4</li>
6 <li>Elemento 5</li>
7 </ul>

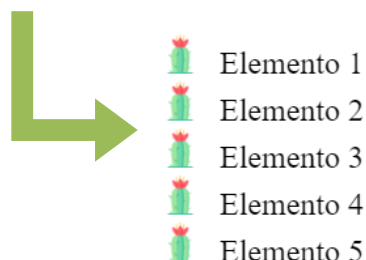
```

CSS

```

1 ul {
2     list-style: none; /* Eliminamos los marcadores predeterminados */
3     padding: 0; /* Eliminamos el relleno predeterminado */
4 }
5
6 li::before {
7     content: ""; /* Dejamos el contenido en blanco */
8     display: inline-block;
9     width: 20px; /* Ancho de la imagen de la viñeta */
10    height: 20px; /* Alto de la imagen de la viñeta */
11    background-image: url('https://cdn-icons-png.flaticon.com/128/4139/4139468.png');
12    /* Ruta de la imagen de la viñeta */
13    background-size: contain; /* Ajustar la imagen al contenedor */
14    margin-right: 10px; /* Espacio entre la viñeta y el texto del elemento */
15 }

```





3. Estilos en formularios

A las etiquetas de los formularios se les puede dar estilo CSS al igual que al resto de elementos de nuestro documento. Sin embargo, debemos conocer una serie de propiedades que nos ayuden a elaborar nuestros diseños de la forma más óptima posible.

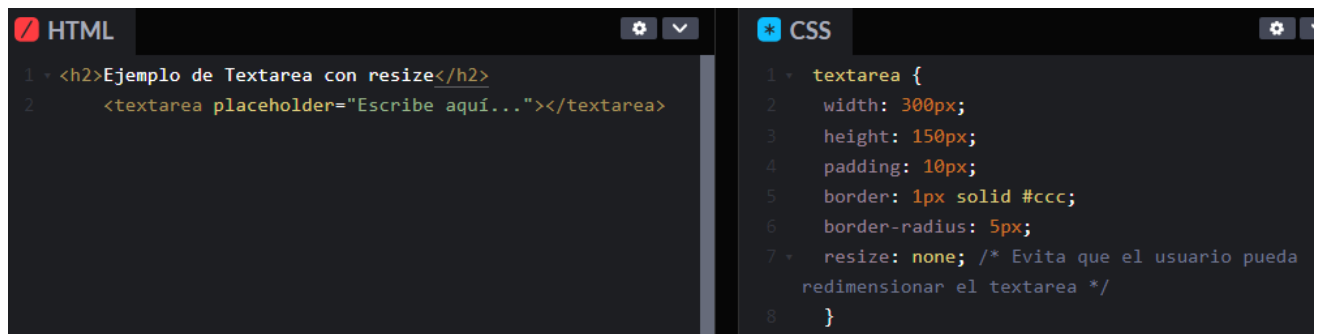
3.1 Propiedad `resize`

Por defecto, los elementos `<textarea>` permiten cambiar el tamaño por el usuario. Podemos anular este comportamiento utilizando la propiedad `resize:none`.

Los **valores** que puede tomar esta propiedad son los que se reflejan a continuación:

Valores	Descripción
<code>none</code>	No permite cambiar el tamaño del elemento
<code>horizontal</code>	Permite cambiar el tamaño del elemento horizontalmente
<code>vertical</code>	Permite cambiar el tamaño del elemento verticalmente
<code>both</code>	Permite cambiar el tamaño del elemento en horizontal y vertical

En el siguiente ejemplo, el `<textarea>` no permite que el usuario cambie su tamaño. Se puede ajustar la propiedad `resize` según nuestras necesidades.



Ejemplo de Textarea con `resize`

Escribe aquí...



3.2 Estilos en el input: pseudo-clase :focus

Cuando un usuario interactúa con un input, ya sea para ingresar texto o seleccionar una opción, **es importante proporcionar retroalimentación visual** para mejorar la experiencia del usuario. Una forma de lograr esto es aplicando estilos específicos cuando el input está en estado **:focus**.

El pseudo-clase **:focus** se utiliza para aplicar estilos a un elemento cuando recibe el foco de atención. Por **ejemplo**, cuando un usuario hace clic en un input, este entra en estado **:focus**. Veamos algunas técnicas comunes para estilizar inputs en estado **:focus**.

3.2.1 Eliminar el contorno predeterminado

Por defecto, los navegadores suelen agregar un contorno alrededor del input cuando está en estado **:focus**. Este contorno puede no ajustarse al diseño de tu sitio web o aplicación. Para eliminarlo, podemos usar la propiedad **outline**:

```
input:focus {  
  outline: none; }
```

3.2.2 Estilizar el fondo y el borde

Puedes personalizar el fondo y el borde del input cuando está en estado **:focus** para resaltar su estado activo. Por ejemplo:

```
input:focus {  
  background-color: #f0f0f0;  
  border: 1px solid #ccc;  
}
```

3.2.3 Animaciones y transiciones

Agregar animaciones o **transiciones** sutiles puede hacer que la interacción con los inputs sea más atractiva. Por ejemplo, podemos animar el cambio de color del borde cuando el input está en estado **:focus**:



```
input {  
  transition: border-color 0.3s ease; }  
input:focus {  
  border-color: #007bff;}
```

3.3 Estilos para inputs

Los estilos en los inputs pueden mejorar significativamente la estética y la experiencia de usuario de tus formularios. Veamos algunos estilos creativos que podemos aplicar para personalizar nuestros inputs.

3.3.1 Estilo básico

El estilo básico es simple y funcional, ideal para una apariencia clásica y limpia.

```
<input type="text" class="basic-input" placeholder="Nombre">
```

```
.basic-input {  
  border: 1px solid #ccc;  
  border-radius: 5px;  
  padding: 10px;  
  width: 250px;  
}
```



Nombre

3.3.2 Bordes redondeados

Los bordes redondeados pueden hacer que tus inputs se vean **más suaves y modernos**.

```
<input type="text" class="rounded-input" placeholder="Correo electrónico">
```




```
.rounded-input {  
  border: 2px solid #3498db;  
  border-radius: 20px;  
  padding: 12px;  
  width: 300px;  
}
```



Correo electrónico

3.3.3 Sombras

Aplicar **sombras** al fondo del input puede agregar profundidad y estilo.

```
<input type="text" class="shaded-input" placeholder="Contraseña">
```

```
.shaded-input {  
  background-color: #f2f2f2;  
  border: none;  
  padding: 10px;  
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);  
  width: 280px;  
}
```



Contraseña



3.3.4 Hover y focus

Agregar efectos de hover y focus puede mejorar la interactividad y la retroalimentación para el usuario.

```
<input type="text" class="hover-focus-input" placeholder="Teléfono">
```

```
.hover-focus-input {  
  border: 1px solid #ccc;  
  padding: 10px;  
  transition: border-color 0.3s;  
  width: 250px;  
}  
  
.hover-focus-input:hover,  
.hover-focus-input:focus {  
  border-color: #66afe9;  
}
```



Teléfono

3.3.5 Iconos en inputs

Incorporar iconos o etiquetas dentro del input puede mejorar la usabilidad y el diseño general. A continuación, se muestra un ejemplo:

Código HTML:

```
<div class="input-container">  
  <i class="material-icons input-icon">person</i>  
  <input type="text" class="icon-input" placeholder="Nombre">  
</div>
```



```
<div class="input-container">
  <i class="material-icons input-icon">email</i>
  <input type="text" class="icon-input" placeholder="Correo
    electrónico">
</div>
<div class="input-container">
  <i class="material-icons input-icon">lock</i>
  <input type="password" class="icon-input" placeholder="Contraseña">
</div>
```

Código CSS:

```
.input-container {
  position: relative;
  margin-bottom: 20px; }
.icon-input {
  padding: 10px 15px 10px 45px;
  border: 2px solid #2196F3;
  border-radius: 25px;
  width: 300px;
  font-size: 16px;
  color: #333;
  background-color: #f7f7f7;
  outline: none; }
.input-icon {
  position: absolute;
  top: 50%;
  left: 15px;
  transform: translateY(-50%);
  color: #777;
  font-size: 20px; }
```



 Nombre

 Correo electrónico

 Contraseña

3.3.6 Animaciones en inputs

La inclusión de animaciones sutiles puede elevar la interactividad y el atractivo visual de tus inputs. Un ejemplo claro es cuando aplicamos una **transición** hacer foco en el input, expandiéndolo ligeramente para enfatizar su estado activo. Veamos cómo implementarlo:

```
<input type="text" class="animated-input" placeholder="Buscar">
```

```
.animated-input {  
  padding: 10px;  
  width: 250px;  
  border: 1px solid #ccc;  
  transition: width 0.3s;  
}  
  
.animated-input:focus {  
  width: 300px;  
}
```





3.4 Estilos para inputs requeridos, asterisco en label

Veamos ahora cómo aplicar estilos específicos a los inputs que son marcados como requeridos en el formulario. Esto se hace mediante la adición de un indicativo, por ejemplo un asterisco, al final de las etiquetas `<label>` correspondientes a los campos obligatorios. Estos estilos ayudan a resaltar visualmente los campos que deben ser completados por el usuario.

En el siguiente ejemplo se usa el **pseudoelemento** `:after` y la propiedad `content` para insertar un asterisco al final de los label que tienen la clase `required`.

```
label.required::after {  
    content: "*";  
    color: red;  
    margin-left: 5px;  
}
```

```
HTML  
1 <form>  
2   <div>  
3     <label for="nombre">Nombre:</label>  
4     <input type="text" id="nombre">  
5   </div>  
6   <div>  
7     <label for="email" class="required">Correo Electrónico:  
8     </label>  
9     <input type="email" id="email">  
10  </div>  
11  <div>  
12    <label for="mensaje" class="inline">Mensaje:</label>  
13    <textarea id="mensaje" rows="4" cols="50"></textarea>  
14  </div>  
15  <button type="submit">Enviar</button>  
16 </form>
```



```
* CSS
1 + body {
2   font-family: Arial, sans-serif;
3   background-color: #f2f2f2;
4   margin: 0;
5   padding: 20px;
6 }
7 + form {
8   background-color: #fff;
9   border-radius: 10px;
10  padding: 20px;
11  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
12 }
13
14 + label {
15   display: block;
16   font-weight: bold;
17   margin-bottom: 5px;
18   color: #333;
19   font-size: 16px;
20   padding-bottom: 5px;
21   font-family: 'Arial', sans-serif;
22 }
23 + label.required::after {
24   content: "**";
25   color: red;
26   margin-left: 5px;
27 }
28 + label.inline {
29   display: inline-block;
30   margin-right: 10px;
31 }
32 input[type="text"],
33 input[type="email"],
34 + textarea {
35   width: 100%;
36   padding: 10px;
37   margin-bottom: 15px;
38   border: 1px solid #ccc;
39   border-radius: 5px;
40   box-sizing: border-box;
41 }
42
43 input[type="text"]:focus,
44 input[type="email"]:focus,
45 + textarea:focus {
46   border-color: #007bff;
47   outline: none;
48 }
49 + button {
50   padding: 10px 20px;
51   background-color: #007bff;
52   border: none;
53   color: #fff;
54   border-radius: 5px;
55   cursor: pointer;
56   font-size: 16px;
57 }
58 + button:hover {
59   background-color: #0056b3;
60 }
```



Nombre:

Correo Electrónico: *

Mensaje:



4. Cursores en CSS.

En CSS, los cursores se utilizan para **cambiar la apariencia del cursor** del ratón cuando se encuentra **sobre un elemento interactivo**. Esto permite personalizar la forma del cursor y proporcionar retroalimentación visual al usuario.

CSS proporciona una variedad de valores para la propiedad **cursor**, que se utiliza para especificar el estilo del cursor. Por **ejemplo**:

```
.div {  
    cursor: crosshair;  
}
```

En la siguiente tabla se pueden ver algunos **ejemplos de diferentes cursores** en CSS:

Valor	Descripción
crosshair	Cursor tipo cruz para tareas en las que requieres precisión.
help	Interrogación. Cursor de ayuda.
move	Flechas hacia todos lados. Cursor para mover elementos.
pointer	Mano o apuntador. Cursor para hacer clic.
progress	Barra progreso o similar. Cursor que indica que se está trabajando en segundo plano.
text	Cursor que permite seleccionar texto.
wait	Cursor que indica que se debe esperar.
vertical-text	Flecha vertical, utilizado para texto en orientación vertical.
alias	Flecha diagonal, utilizado para indicar que se puede acceder a un enlace o recurso externo.
no-drop	Círculo con una línea diagonal, utilizado para indicar que no se permite soltar el elemento arrastrado.
grabbing	Mano cerrada, utilizado para indicar que el elemento se puede agarrar.
grab	Mano abierta, utilizado para indicar que el elemento está siendo arrastrado o agarrado.
zoom-in	Símbolo de lupa con un signo de más, utilizado para indicar que se puede hacer zoom in.
zoom-out	Símbolo de lupa con un signo de menos, utilizado para indicar que se puede hacer zoom out.

not-allowed	Círculo con una línea diagonal, utilizado para indicar que la interacción no está permitida.
cell	Cursor tipo cruz, utilizado en hojas de cálculo para ajustar celdas.
copy	Puntero con un símbolo de copia, utilizado para indicar que se puede realizar una copia de un elemento.
ew-resize	Flecha bidireccional horizontal, utilizado para indicar que se puede ajustar el tamaño horizontalmente.
ns-resize	Flecha bidireccional vertical, utilizado para indicar que se puede ajustar el tamaño verticalmente.
nwse-resize	Flecha bidireccional diagonal, utilizado para indicar que se puede ajustar el tamaño diagonalmente en dirección noroeste-sureste.
nesw-resize	Flecha bidireccional diagonal, utilizado para indicar que se puede ajustar el tamaño diagonalmente en dirección noreste-suroeste.
col-resize	Flecha bidireccional horizontal con una línea vertical, utilizado para indicar que se puede ajustar el tamaño de una columna.
row-resize	Flecha bidireccional vertical con una línea horizontal, utilizado para indicar que se puede ajustar el tamaño de una fila.
context-menu	Puntero con un símbolo de menú, utilizado para indicar que se puede abrir un menú contextual.
none	No se muestra ningún cursor.
personalized	Imagen o valor personalizado en el cursor.

A continuación, se muestra la apariencia de algunos de ellos:

I auto	↕ move	👉 no-drop	↕ col-resize
↕ all-scroll	👉 pointer	🚫 not-allowed	↕ row-resize
+ crosshair	🕒 progress	↔ e-resize	↗ ne-resize
👉 default	I text	↑ n-resize	↖ nw-resize
👉 help	↔ vertical-text	↓ s-resize	↘ se-resize
I inherit	🕒 wait	↔ w-resize	↙ sw-resize

Estas explicaciones proporcionan una descripción breve de cada cursor, pero es posible encontrar más detalles y variaciones de estos cursores en la especificación CSS y en la documentación de los navegadores. Es importante tener en cuenta que **algunos valores de cursor pueden no ser compatibles en todos los navegadores** o en todos los elementos.

Veamos cómo se visualizan los cursores en un **ejemplo práctico**:

https://www.w3schools.com/cssref/tryit.php?filename=trycss_cursor



5. Definición de propiedades personalizadas

Las **CSS Custom Properties** o **propiedades personalizadas** (también llamadas variables CSS) nos permiten **definir nombres para valores determinados de las propiedades**. De esta forma, se puede **utilizar ese nombre como si fuese una variable**. La gran ventaja es que al modificar el valor de esas “variables” se cambia automáticamente en todos los lugares del documento CSS en los que se haya usado ese nombre.

Esta característica es **utilizada recientemente** ya que no estaba presente en las primeras versiones de CSS ni soportada en todos los navegadores en las siguientes versiones.

Para definir una **propiedad personalizada** pondremos **dos guiones** `--` delante del nombre que queramos utilizar.

Para definir la propiedad personalizada para todo el documento HTML la incluiremos en el **pseudoelemento :root**. Ejemplo:

```
:root {  
  --principal-color: black; /* Valor personalizado */  
  --font: "Arial";  
}
```

5.1 Uso de propiedades personalizadas

Para utilizar una **propiedad personalizada** hay que insertar su nombre dentro de la expresión `var()`:

```
.clase {  
  background-color: var(--principal-color);  
  font-family: var(--font);  
}
```



Ejemplo:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      :root {
        --principal-color: brown;
        --secondary-color: #e1f4fc;
        --font: "Arial";
      }
      p { color: var(--principal-color);}
      div { background-color: var(--secondary-color);
            font-family: var(--font);}
    </style>
  </head>
  <body>
    <h1>CSS Custom Properties</h1>
    <p>Este texto se ve de color marrón</p>
    <div>Este fondo se ve verde azul y el texto en
    Arial</div>
    <span>Este texto no tiene ningún estilo definido</
    span>
  </body>
</html>
```

CSS Custom Properties

Este texto se ve de color marrón

Este fondo se ve verde azul y el texto en Arial
Este texto no tiene ningún estilo definido



6. Funciones matemáticas

CSS proporciona varias funciones matemáticas que se pueden utilizar para **realizar cálculos** y **manipular valores numéricos** en propiedades CSS. Algunas de las funciones matemáticas más comunes en CSS son las siguientes:

Propiedad	Descripción	Valores Ejemplo
<code>abs()</code>	Devuelve el valor absoluto de un número.	<code>abs(-10) = 10</code>
<code>sin()</code>	Devuelve el seno de un ángulo.	<code>sin(45deg) ≈ 0.71</code>
<code>cos()</code>	Devuelve el coseno de un ángulo.	<code>cos(60deg) = 0.5</code>
<code>tan()</code>	Devuelve la tangente de un ángulo.	<code>tan(30deg) ≈ 0.58</code>
<code>sqrt()</code>	Devuelve la raíz cuadrada de un número.	<code>sqrt(16) = 4</code>
<code>pow()</code>	Eleva un número a una potencia.	<code>pow(2, 3) = 8</code>
<code>min()</code>	Devuelve el valor mínimo entre dos números.	<code>min(10, 5) = 5</code>
<code>max()</code>	Devuelve el valor máximo entre dos números.	<code>max(10, 5) = 10</code>
<code>random()</code>	Devuelve un número aleatorio entre 0 y 1.	<code>random() ≈ 0.35</code>
<code>floor()</code>	Devuelve el valor entero menor o igual.	<code>floor(3.8) = 3</code>
<code>ceil()</code>	Devuelve el valor entero mayor o igual.	<code>ceil(3.2) = 4</code>
<code>round()</code>	Devuelve el valor redondeado al entero más cercano.	<code>round(3.7) = 4</code>
<code>clamp()</code>	Limita un valor dentro de un rango. <code>clamp(<min>, <ideal>, <max>):</code> establece un valor entre un límite superior y uno inferior, basado en un valor ideal establecido.	<code>clamp(4, 5, 8) = 5</code> <code>clamp(320px, 90%, 48rem)</code>
<code>calc()</code>	Realiza cálculos matemáticos en propiedades CSS.	<code>width: calc(100% - 20px)</code>
<code>attr()</code>	Obtiene el valor de un atributo HTML y lo utiliza en CSS.	<code>content: attr(data-text)</code>

Se pueden ver más funciones y ejemplos en: <https://lenguajecss.com/css/unidades-css/funciones-css/>.



6.1 calc()

La función matemática **calc()** en CSS se utiliza para realizar cálculos aritméticos y combinar diferentes unidades de medida. Aquí tienes algunos ejemplos de cómo se puede utilizar:

- **Ejemplo 1: Suma de valores.** En este ejemplo, se realiza la suma de 100px y 20px, lo que da como resultado un ancho de 120px.

```
width: calc(100px + 20px); /* Resultado: 120px */
```

- **Ejemplo 2: Resta de valores.** En este caso, se resta 10px al 80% del ancho disponible, lo que dará como resultado un ancho relativo.

```
width: calc(80% - 10px); /* Resultado: El 80% del ancho disponible  
menos 10px */
```

- **Ejemplo 3: Multiplicación de valores.** Aquí se realiza la multiplicación de 2 por el 50% del ancho disponible, obteniendo un ancho relativo igual al doble del 50%.

```
width: calc(2 * 50%); /* Resultado: El doble del 50% del ancho  
disponible */
```

- **Ejemplo 4: Combinación de operaciones.** En este ejemplo, se realiza una combinación de operaciones donde se suma 100px con el 10% del ancho disponible y luego se divide el resultado por 3.

```
width: calc((100px + 10%) / 3); /* Resultado: El resultado de la  
operación combinada */
```

Es posible anidar llamadas a **calc()** dentro de otras llamadas **calc()**.



Ahora veamos un **ejemplo práctico**: Realización de **cálculos con calc()** de CSS para conseguir un contenedor fijo y otro variable.



```
<!DOCTYPE html>
<html>
<head>
<style>
#div1 {
float:left;
width: calc(100% - 230px);
background-color: yellow;
padding: 15px;
margin-bottom: 15px;
clear: both;
box-sizing: border-box;
}
#div2{
float:left;
width: 230px;
padding: 15px;
box-sizing: border-box;
}
</style>
</head>
<body>
<div id="div1">Lorem ipsum dolor sit amet, consectetur
adipiscing elit, sed do eiusmod tempor incididunt ut
labore et dolore magna aliqua. Ut enim ad minim veniam,
quis nostrud exercitation ullamco laboris nisi ut
aliquip ex ea commodo consequat. Duis aute irure dolor
in reprehenderit in voluptate velit esse cillum dolore
eu fugiat nulla pariatur. Excepteur sint occaecat
cupidatat non proident, sunt in culpa qui officia
deserunt mollit anim id est laborum.</div>
<div id="div2">Other text</div>

<div id="div1">Lorem ipsum dolor sit amet, consectetur
adipiscing elit, sed do eiusmod tempor incididunt ut labore
et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud
exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in
voluptate velit esse cillum dolore eu fugiat nulla pariatur.
Excepteur sint occaecat cupidatat non proident, sunt in culpa
qui officia deserunt mollit anim id est laborum.</div>
<div id="div2"><button>Button</button></div>

<div id="div1">Lorem ipsum dolor sit amet, consectetur
adipiscing elit, sed do eiusmod tempor incididunt ut labore
et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud
exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in
voluptate velit esse cillum dolore eu fugiat nulla pariatur.
Excepteur sint occaecat cupidatat non proident, sunt in culpa
qui officia deserunt mollit anim id est laborum.</div>
<div id="div2">
</div>
</body>
</html>
```

Other text

Button

uno

6.1.1 CSS calc() para mejorar la tipografía fluida

Una técnica para llevar a cabo una tipografía responsive es el uso de la función `calc()` en CSS, que permite **realizar cálculos matemáticos** para determinar el tamaño de la fuente. Por ejemplo, podemos combinar `calc()` con unidades relativas.

En el siguiente **ejemplo**, el tamaño de la fuente será de al menos 16 píxeles, pero también aumentará en un 0.5% del ancho de la pantalla.

```
font-size: calc(16px + 0.5vw);
```



6.2 min()

La función matemática **min()** en CSS se utiliza para obtener el **valor mínimo entre varios valores** o dimensiones. A continuación, se pueden ver algunos **ejemplos** de cómo se puede utilizar:

- **Ejemplo 1: Obtener el valor mínimo entre dos anchos.** En este ejemplo, se obtiene el valor mínimo entre 200px y el 50% del ancho disponible, lo que dará como resultado el valor más pequeño entre ambos.

```
width: min(200px, 50%); /* Resultado: El valor mínimo entre  
200px y el 50% del ancho disponible */
```

- **Ejemplo 2: Obtener el valor mínimo entre tres alturas.** Aquí se obtiene el valor mínimo entre 100px, 150px y el 50% del alto disponible, resultando en la altura más pequeña entre los tres valores.

```
height: min(100px, 150px, 50%); /* Resultado: El valor mínimo  
entre 100px, 150px y el 50% del alto disponible */
```

- **Ejemplo 3: Combinar valores absolutos y relativos.** En este caso, se obtiene el valor mínimo entre 16px y 1.2 veces el tamaño de fuente actual, lo que dará como resultado el tamaño de fuente más pequeño entre ambos.

```
font-size: min(16px, 1.2em); /* Resultado: El valor mínimo entre  
16px y 1.2 veces el tamaño de fuente actual */
```

6.3 max()

La función matemática **max()** en CSS se utiliza para obtener el valor máximo entre varios valores o dimensiones. A continuación, se muestran algunos **ejemplos** de cómo se puede utilizar:

- **Ejemplo 1: Obtener el valor máximo entre dos anchos.** En este ejemplo, se obtiene el valor máximo entre 200px y el 50% del ancho disponible, lo que dará como resultado el valor más grande entre ambos.



```
width: max(200px, 50%); /* Resultado: El valor máximo entre 200px  
y el 50% del ancho disponible */
```

- **Ejemplo 2: Obtener el valor máximo entre tres alturas.** Aquí se obtiene el valor máximo entre 100px, 150px y el 50% del alto disponible, resultando en la altura más grande entre los tres valores.

```
height: max(100px, 150px, 50%); /* Resultado: El valor máximo  
entre 100px, 150px y el 50% del alto disponible */
```

- **Ejemplo 3: Combinar valores absolutos y relativos.** En este caso, se obtiene el valor máximo entre 16px y 1.2 veces el tamaño de fuente actual, lo que dará como resultado el tamaño de fuente más grande entre ambos.

```
font-size: max(16px, 1.2em); /* Resultado: El valor máximo entre  
16px y 1.2 veces el tamaño de fuente actual */
```

`min()`, `max()`, `clamp()` pueden utilizarse junto con `calc()` y además pueden anidarse, es decir, utilizarse combinadas:

`calc(min())`, `max(calc())`, `calc(max(min()))`, etc.



6.4 round()

La función matemática `round()` en CSS se utiliza para redondear un número al entero más cercano. Aquí tienes algunos ejemplos de cómo se puede utilizar:

- **Ejemplo 1: Redondear un número decimal.** En este ejemplo el número decimal 3.7rem se redondea al entero más cercano, que es 4rem.

```
width: round(3.7rem); /* Resultado: 4rem */
```




- **Ejemplo 2: Redondear un número negativo.** Aquí, el número negativo -2.3em se redondea al entero más cercano, que es -2em.

```
padding: round(-2.3em); /* Resultado: -2em */
```

- **Ejemplo 3: Redondear un número con decimales a un número específico de lugares decimales.** En este caso, el número 1.2345 se redondea a 2 lugares decimales, resultando en 1.23.

```
font-size: round(1.2345, 2); /* Resultado: 1.23 */
```

6.5 attr()

La función matemática `attr()` en CSS se utiliza para obtener el valor de un atributo HTML y utilizarlo en propiedades CSS. Aquí tienes algunos ejemplos de cómo se puede utilizar:

- **Ejemplo 1: Obtener el valor de un atributo y utilizarlo como ancho.** En este ejemplo, se obtiene el valor del atributo `data-width` y se utiliza como valor para la propiedad `width`. Esto puede ser útil cuando se necesita establecer dinámicamente el ancho de un elemento basado en un atributo específico.

```
width: attr(data-width);
```

- **Ejemplo 2: Obtener el valor de un atributo y utilizarlo en una operación matemática.** Aquí, se obtiene el valor del atributo `data-height` y se utiliza en una operación matemática dentro de la función `calc()`. Esto permite realizar cálculos con el valor del atributo y aplicar el resultado como valor para la propiedad `height`.

```
height: calc(attr(data-height) * 2);
```

- **Ejemplo 3: Obtener el valor de un atributo y utilizarlo como contenido de un pseudo-elemento.** En este caso, se obtiene el valor del atributo `data-content` y se utiliza como contenido para un pseudo-elemento. Esto permite mostrar dinámicamente el contenido de un atributo HTML en el estilo de un elemento.



content: attr(data-content);

Veamos dos **ejemplos** prácticos: en el primero utilizamos el atributo *data-author* para mostrarlo como contenido en el documento HTML, y en el segundo insertaremos el valor del atributo *href* después de cada link utilizando para ello el pseudoelemento *::after*.

1

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      .element::after {
        content: attr(data-author);
        font-weight: bold;
        color: green;
      }
    </style>
  </head>

  <body>
    <p class="element" data-author="Andrea">El autor es </p>
  </body>
</html>
```

El autor es **Andrea**

2

```
<!DOCTYPE html>
<html>
<head>
<style>
  a::after {
    content: " (" attr(href) ")";
    color: green;
  }
  a { text-decoration: none; }
</style>
</head>
<body>

<h1>La función attr() </h1>
<p>Inserta el valor del atributo href entre paréntesis después de
cada link:</p>
<p><a href="https://www.w3schools.com">Visita W3Schools</a></p>
<p><a href="https://www.w3schools.com/html/">Visita W3Schools'
HTML tutorial</a></p>
</body>
</html>
```

La función attr()

Inserta el valor del atributo href entre paréntesis después de cada link:

Visita W3Schools (<https://www.w3schools.com>)

Visita W3Schools' HTML tutorial (<https://www.w3schools.com/html/>)



6.6 CSS clamp(): tipografía fluida o responsive

La función CSS `clamp()` nos permite definir un **rango de valores** para el tamaño de la fuente, incluyendo un **tamaño mínimo, máximo y un valor preferido** que se ajusta dinámicamente en relación con el ancho de la pantalla.

Su **sintaxis** básica consiste en tres valores separados por comas:

```
propiedad: clamp(valor_mínimo, valor_preferido, valor_máximo);
```

- 1) **Propiedad**. Es la propiedad a la que se va a aplicar `clamp()`. En el caso de trabajar con tipografías usaremos font-size para modificar el tamaño de las fuentes.
- 2) **valor_mínimo**. Es el tamaño mínimo que se aplicará a la propiedad, independientemente del tamaño de la pantalla o del contenedor. Si el valor preferido es menor que este, se aplicará el valor mínimo.
- 3) **valor_preferido**: Es el valor que se aplicará de manera predeterminada a la propiedad. Este valor puede ser relativo, como en el caso de unidades como `vw` (`viewport width`) o `em`, lo que permite que el tamaño del texto se ajuste dinámicamente en función del tamaño de la pantalla.
- 4) **valor_máximo**: Es el tamaño máximo que se aplicará a la propiedad. Si el valor preferido es mayor que este, se aplicará el valor máximo.

En el siguiente ejemplo, el tamaño de la fuente se ajusta entre 1em y 2.5em, con un tamaño preferido que varía en función del ancho de la pantalla. En el tamaño preferido se usa la **unidad de medida relativa vw o viewport-width**, que no es más que el ancho de la pantalla o **viewport**.

El valor 100vw es el 100% del ancho de la pantalla actual. Por lo que el tamaño de nuestra fuente preferida va a ser un **porcentaje del ancho en pixeles de la pantalla**. Por ejemplo, si decimos que nuestro tamaño preferido es de 3vw, cuando el ancho de la pantalla o viewport sea de 1000px, la fuente tendrá un tamaño de 30px. Si el ancho de la pantalla es de 700px, el tamaño de la fuente será de 21px.

```
HTML CSS
1 <h2>CSS clamp() es la clave para
  implementar la tipografía fluida</h2>
2 h2 {
3   font-size: clamp(1em, 3vw, 2.5em);
4 }
5 @supports not (font-size: clamp(1em, 3vw, 2.5em)) {
6   /* Alternativa para navegadores que no admiten clamp() */
7   font-size: min(max(1em, 3vw), 2.5em);
8 }
```

NOTA. Existen algunas reglas que no hemos visto en clase, y son interesantes. Estas son: `@supports`, `@layer`, `@scope`, y `@container`.

`@supports`: <https://lenguajecss.com/css/compatibilidad-navegadores/la-regla-supports/>

`@layer`: <https://lenguajecss.com/cascada-css/especificidad/regla-layer/>

`@scope`: <https://lenguajecss.com/cascada-css/alcance/la-regla-scope/>

`@container`: <https://lenguajecss.com/css/responsive-web-design/css-container-queries/>

6.7 CSS clamp() junto con calc() para cálculos matemáticos y establecer límites

Es común mezclar el uso de `calc()` y `clamp()` en CSS para crear estilos más flexibles y dinámicos. Por ejemplo, podrías utilizar `calc()` para **realizar cálculos matemáticos** en propiedades como el ancho, el margen o el padding de un elemento, y luego usar `clamp()` para **establecer límites** para esas propiedades.

```
/* Usando calc() y clamp() juntos para el tamaño de fuente del texto */
.texto {
  font-size: clamp(12px, calc(1rem + 3vw), 36px);

  /* En este ejemplo, el tamaño de la fuente del 'texto' se ajustará
  dinámicamente entre 12px y 36px, pero nunca será menor que el tamaño
  base de 1rem más el 3% del ancho de la ventana. */
}
```

En este caso, el tamaño de fuente del texto variará entre 12px y 36px, pero nunca será menor que el tamaño base de 1rem más el 3% del ancho de la ventana (`vw`). Esto significa que a medida que el ancho de la ventana aumenta, el tamaño de la fuente del texto también aumentará, proporcionando así un diseño adaptativo y escalable



7. CSS line-clamp(): acortar texto a un nº de líneas.

La propiedad **CSS line-clamp** nos permite **acortar un texto a un número específico de líneas**, lo que resulta especialmente útil en interfaces de usuario donde el espacio es limitado. Además de truncar el texto, **line-clamp** se combina con otras propiedades CSS, como **display** y **overflow**, para ajustar el comportamiento del texto, incluyendo la adición de puntos suspensivos, también llamados puntos de elipsis, al final del texto truncado.

7.1 Cómo implementar line-clamp en CSS

Para **limitar** un texto a un **número concreto de líneas** le aplicaremos las siguientes **propiedades**:

- **display: -webkit-box;**. Esta propiedad se utiliza para establecer el tipo de caja de diseño utilizado para el elemento. En este caso, **-webkit-box** es necesario para **compatibilidad con navegadores basados en WebKit**, como Safari y Chrome.
- **-webkit-line-clamp: <numero_de_lineas>;**. Esta es la parte principal de la propiedad **line-clamp**. Aquí se especifica el número de líneas que se mostrarán antes de truncar el texto. Por ejemplo, si establecemos **-webkit-line-clamp: 3;**, solo se mostrarán las primeras tres líneas del texto y el resto será truncado.
- **-webkit-box-orient: horizontal | vertical;**. Esta propiedad se utiliza para establecer la orientación del contenido dentro del contenedor. En el contexto de **line-clamp**, normalmente se usa **vertical** para que los elementos de línea se apilen verticalmente dentro del contenedor flexible, permitiendo así la truncación vertical del texto.
 - **horizontal**. Los elementos se apilan horizontalmente.
 - **vertical**. Los elementos se apilan verticalmente.
- **overflow: hidden;**. Esta propiedad se utiliza para ocultar el contenido que se desborda del contenedor. Es importante para asegurarse de que el texto truncado no sea visible fuera del área designada.



Por tanto, su **sintaxis** será la siguiente:

```
.elemento-truncado {  
    display: -webkit-box;  
    -webkit-line-clamp: <numero_de_lineas>;  
    -webkit-box-orient: horizontal | vertical;  
    overflow: hidden; }
```

Ejemplo de implementación: Supongamos que queremos truncar un párrafo después de mostrar solo 3 líneas de texto. Podemos aplicar **line-clamp** de la siguiente manera:

```
.parrafo-truncado {  
    display: -webkit-box;  
    -webkit-line-clamp: 3;  
    -webkit-box-orient: vertical;  
    overflow: hidden;  
}
```

En este ejemplo, cualquier texto dentro de un elemento con la clase `.parrafo-truncado` se truncará después de mostrar solo 3 líneas, y los tres puntos suspensivos (...) se añadirán automáticamente para indicar que el contenido ha sido truncado. Esto es útil para mostrar fragmentos de texto en interfaces de usuario donde el espacio es escaso.

7.2 Ejemplos prácticos de line-clamp

La propiedad **line-clamp** es una herramienta versátil que puede aplicarse en una variedad de situaciones en el diseño web. A continuación, se presentan algunos ejemplos prácticos de cómo utilizar **line-clamp** para mejorar la legibilidad y el diseño de tu contenido.

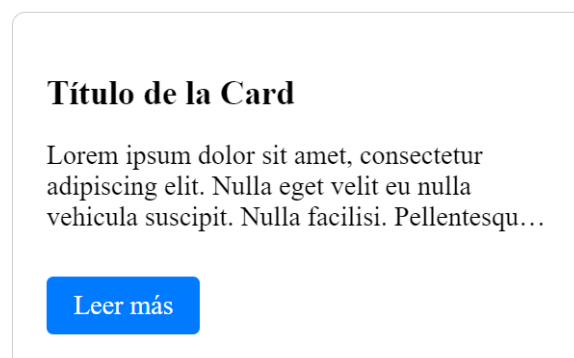


7.2.1 Ejemplo 1. Truncado de texto en cards o tarjetas de contenido

En tarjetas de contenido, como aquellas utilizadas en diseño de mosaicos o galerías, puedes utilizar **line-clamp** para truncar el texto que acompaña a cada tarjeta después de un número específico de líneas. Esto permite mantener un diseño uniforme y consistente en toda la presentación de tarjetas, proporcionando una experiencia visualmente atractiva y fácil de leer para los usuarios.

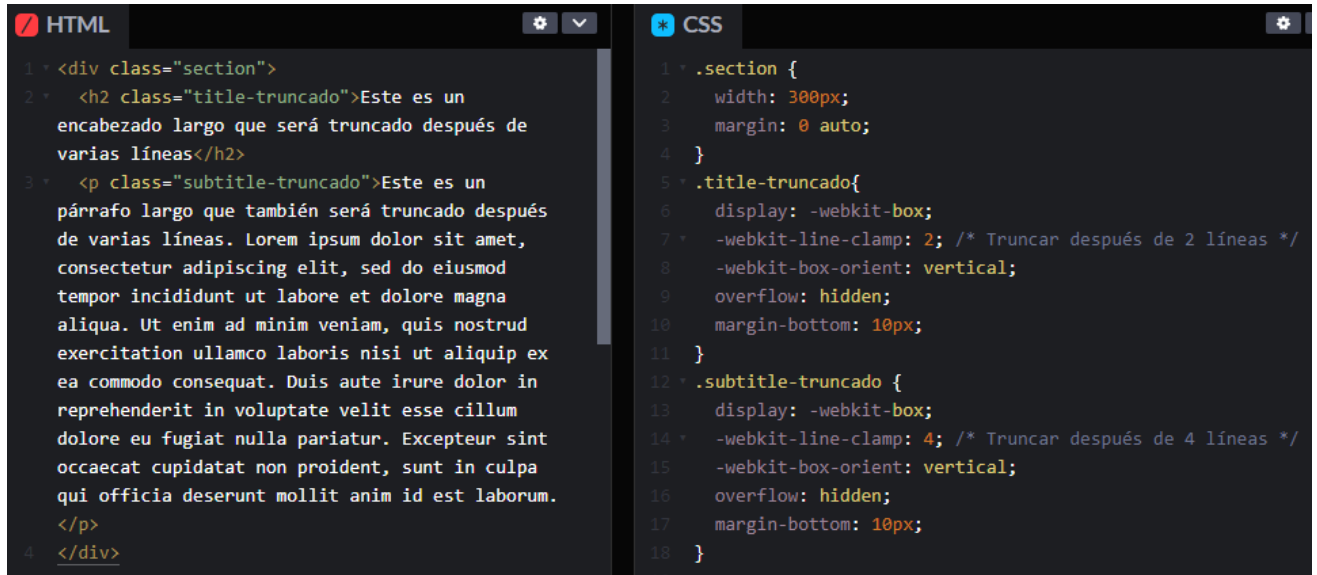
```
HTML
1 <div class="card">
2   <h2 class="card-title">Título de la Card</h2>
3   <p class="card-text">
4     Lorem ipsum dolor sit amet, consectetur
5     adipiscing elit. Nulla eget velit eu nulla vehicula
6     suscipit. Nulla facilisi.
7     Pellentesque habitant morbi tristique senectus et
8     netus et malesuada fames ac turpis egestas. Integer non
9     eros eu eros bibendum vehicula.
10  </p>
11  <a href="#" class="btn">Leer más</a>
12 </div>

CSS
1 .card {
2   border: 1px solid #ccc;
3   border-radius: 8px;
4   padding: 20px;
5   max-width: 300px;
6 }
7 .card-title {
8   font-size: 20px;
9   margin-bottom: 10px;
10 }
11 .card-text {
12   display: -webkit-box;
13   -webkit-line-clamp: 3;
14   -webkit-box-orient: vertical;
15   overflow: hidden;
16 }
17 .btn {
18   background-color: #007bff;
19   color: #fff;
20   padding: 8px 16px;
21   border-radius: 4px;
22   text-decoration: none;
23   display: inline-block;
24   margin-top: 10px;
25 }
```



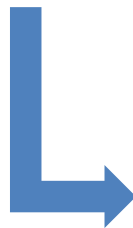
7.2.2 Ejemplo 2. Acortar textos de los encabezados o subtítulos

En secciones de una página web donde los encabezados o subtítulos pueden ser demasiado largos, puedes utilizar `line-clamp` para truncarlos después de un cierto número de líneas. Esto ayuda a mantener un diseño limpio y evitar que los encabezados dominen visualmente la página.



```
HTML
1 <div class="section">
2   <h2 class="title-truncado">Este es un
   encabezado largo que será truncado después de
   varias líneas</h2>
3   <p class="subtitle-truncado">Este es un
   párrafo largo que también será truncado después
   de varias líneas. Lorem ipsum dolor sit amet,
   consectetur adipiscing elit, sed do eiusmod
   tempor incididunt ut labore et dolore magna
   aliqua. Ut enim ad minim veniam, quis nostrud
   exercitation ullamco laboris nisi ut aliquip ex
   ea commodo consequat. Duis aute irure dolor in
   reprehenderit in voluptate velit esse cillum
   dolore eu fugiat nulla pariatur. Excepteur sint
   occaecat cupidatat non proident, sunt in culpa
   qui officia deserunt mollit anim id est laborum.
   </p>
4 </div>

CSS
1 .section {
2   width: 300px;
3   margin: 0 auto;
4 }
5 .title-truncado {
6   display: -webkit-box;
7   -webkit-line-clamp: 2; /* Truncar después de 2 líneas */
8   -webkit-box-orient: vertical;
9   overflow: hidden;
10  margin-bottom: 10px;
11 }
12 .subtitle-truncado {
13   display: -webkit-box;
14   -webkit-line-clamp: 4; /* Truncar después de 4 líneas */
15   -webkit-box-orient: vertical;
16   overflow: hidden;
17   margin-bottom: 10px;
18 }
```



**Este es un encabezado largo
que será truncado después...**

Este es un párrafo largo que también será
truncado después de varias líneas. Lorem
ipsum dolor sit amet, consectetur adipiscing
elit, sed do eiusmod tempor incididunt ut...

7.2.3 Ejemplo 3. Acortar textos de productos en una tienda online

En una tienda online, puedes utilizar `line-clamp` para truncar las descripciones largas de productos después de un cierto número de líneas. Esto permite a los usuarios obtener una vista previa del contenido sin abrumarlos con información excesiva.



```
HTML
1 <div class="product">
2   
3   <div class="product-info">
4     <h3 class="product-title">Nombre del Producto</h3>
5     <p class="product-description">Esta es una
   descripción larga del producto que será truncada después
   de varias líneas. La descripción proporciona información
   detallada sobre el producto, pero al truncarla,
   mantenemos un diseño limpio y evitamos abrumar a los
   usuarios con demasiada información.</p>
6     <button class="add-to-cart">Añadir al
   Carrito</button>
7   </div>
8 </div>

CSS
1 .product {
2   display: flex;
3   align-items: center;
4   width: 300px; /* Ancho del contenedor del producto */
5   margin: 0 auto; /* Centrar el contenedor horizontalmente */
6   border: 1px solid #ccc;
7   border-radius: 8px;
8   padding: 20px;
9   margin-bottom: 20px; }
10 .product img {
11   width: 100px; /* Ancho de la imagen del producto */
12   margin-right: 20px; }
13 .product-info {
14   flex: 1; }
15 .product-title {
16   margin-top: 0;
17   margin-bottom: 10px;
18   font-size: 18px;
19   font-weight: bold; }
20
21 .product-description {
22   display: -webkit-box;
23   -webkit-line-clamp: 3; /* Truncar después de 3 líneas */
24   -webkit-box-orient: vertical;
25   overflow: hidden;
26   margin: 0; }
27
28 .add-to-cart {
29   background-color: #007bff;
30   color: #fff;
31   border: none;
32   border-radius: 5px;
33   padding: 10px 20px;
34   font-size: 16px;
35   cursor: pointer;
36   margin-top: 10px; }
37 .add-to-cart:hover {
38   background-color: #0056b3; }
```





8. Transform

Las transformaciones consisten en hacerle **cambios** al diseño a un mismo elemento. Esto no significa que vaya a cambiar su código HTML, sino simplemente la forma en la que se visualiza. Esto es, aplica una **transformación 2D o 3D** a un elemento.

La propiedad *transform* nos permite rotar, escalar, mover, sesgar, etc, elementos.

Antes de usar esta propiedad, habrá que consultar si es **compatible** con el navegador web del usuario y si necesita ir acompañada por un **prefijo**.



Las dos **propiedades** que nos sirven para definir las transformaciones son *transform* y *transform-origin*.

- La propiedad *transform* aplica una o varias transformaciones (rotar, escalar, mover) a un elemento en CSS.
- La propiedad *transform-origin* define el punto exacto alrededor del cual se aplicarán estas transformaciones. Por defecto, el punto de origen es el centro del elemento (50% 50%).

Su **sintaxis** es la siguiente:

```
etiqueta html {  
    transform: none | transform-functions;  
    transform-origin: x-axis y-axis z-axis; }
```

- *x-axis*: define dónde se coloca la vista en el eje x. Posibles valores: left, center, right, una medida de longitud, o un porcentaje (%).
- *y-axis*: define dónde se coloca la vista en el eje y. Posibles valores: top, center, bottom, una medida de longitud, o un porcentaje (%).
- *z-axis*: define dónde se coloca la vista en el eje z. Posibles valores: una medida de longitud.

8.1 Tipos de transformaciones.

Las principales transformaciones (*transform-functions*) que tenemos a nuestro alcance son:

- **rotate**. Nos permite rotar en 2D, los elementos en el sentido de las agujas del reloj. Para ello utilizaremos ángulos (*deg*).

```
transform: rotate(45deg); /* Rota el elemento 45 grados */
```

Con esta transformación se suele utilizar la propiedad '*transform-origin*' que indica el punto de referencia sobre el que va a rotar el elemento, sobre el ejeX, ejeY y ejeZ. Su valor **por defecto es 50% 50% 0**.

A continuación, se presenta un **ejemplo**:

```
<!DOCTYPE html>
<html>
<head>
<style>
#div1 {
  position: relative;
  height: 200px;
  width: 200px;
  margin: 100px;
  padding: 10px;
  border: 1px solid black; }
#div2 {
  padding: 50px;
  position: absolute;
  border: 1px solid black;
  background-color: red;
  transform: rotate(45deg);
  transform-origin: 20% 40%; }
</style>
</head>
<body>
<h1>The transform-origin
Property</h1>
<div id="div1">
  <div id="div2">HELLO</div>
</div>
</body>
</html>
```

The transform-origin Property

Además de rotar un elemento en 2D, podemos hacerlo en 3D con algunas de las siguientes propiedades:

- [*rotate3d\(x,y,z,angle\)*](#)
- [*rotateX\(angle\)*](#)
- [*rotateY\(angle\)*](#)
- [*rotateZ\(angle\)*](#)



- **skew**. Nos permite '*deformar*' los elementos. Para ello utilizaremos ángulos (*deg*). Esta función se establece con uno o dos valores: `skewX()`, `skewY()` o `skew(x,y)`.

Nota importante: Si se omite el valor sobre el eje y, este será 0 por defecto.

```
transform: skew(45deg); /* Distorsiona el elemento 45 grados  
                        en el eje x, y 0 en el eje y*/
```

- **translate**. Cambia la posición del elemento hacia la **izquierda, derecha, arriba o abajo**. La función `translate()` se establece con uno o dos valores: `translateX()`, `translateY()` o `translate(x,y)`. Los valores x e y son los vectores de translación en las coordenadas x e y. Sus valores pueden estar definidos en píxeles, porcentajes, etc.

Nota importante: Si se omite el valor sobre el eje y, este será 0 por defecto.

```
transform: translate(10px); /* Traslada el elemento 10px hacia  
                           la derecha */
```

Además de cambiar la posición de un elemento en 2D, podemos hacerlo en 3D con algunas de las siguientes propiedades:

- `translate3d(x,y,z)`
- `translateZ(z)`

- **scale**. Nos permite **cambiar la escala** de los elementos, es decir, nos permite **modificar su tamaño**. La función `scale()` se establece con uno o dos valores, que representan la cantidad de escala que se aplica en cada dirección: `scaleX()`, `scaleY()` o `scale(x,y)`.

Se define **mediante un valor numérico** de manera que cuando un valor de coordenadas está fuera del rango [-1, 1], el elemento crece a lo largo de esa dimensión. Cuando está dentro del rango, el elemento se encoge.

Nota importante: Si se omite el valor sobre el eje y, este será igual al del eje x por defecto.



```
transform: scale(0.5); /* Escala el elemento a la mitad tanto  
                        en el eje x como en el eje y */
```

Además de cambiar la escala de un elemento en 2D, podemos hacerlo en 3D con algunas de las siguientes propiedades:

- [scale3d\(x,y,z\)](#)
- [scaleZ\(z\)](#)
- **matrix.** **Mueve** o transforma los **elementos con precisión de píxel**. La función **matrix()** se establece con seis valores numéricos: **matrix (a,b,c,d,x,y)**.

```
matrix(scaleX(), skewY(), skewX(), scaleY(), translateX(), translateY())
```

Los dos últimos valores representan la translación y los primeros la transformación lineal.

Como ya se ha comentado, en las transformaciones **skew**, **scale** y **translate** tenemos **variaciones**, es decir, podemos llevar a cabo una variación para el eje X, otra para el eje Y, o en ambos ejes a la vez:

```
transform: transform-function(cantidad);  
transform: transform-function X(cantidad);  
transform: transform-function Y(cantidad);  
transform: transform-function (cantidadX, cantidadY);  
transform: transform-function () transform-function () transform-  
function ()...;
```



La **diferencia** entre '**translate**' y '**position**' con coordenadas, es que el '**translate**' **no afecta al flujo** de colocación de elementos del resto de cosas que haya en la página web.

Cuando se hace un '**transform**', normalmente es porque está bajo estados de **pseudoclases** (por ejemplo, **:hover**). En este caso, la transición se hace muy brusca, y es



por ello que la propiedad *transform* se suele utilizar junto con la propiedad *transition*, con el objetivo de tener una **transición espaciada en el tiempo**.

```
.caja1{
  -webkit-transition: 1s linear;
  transition: 1s linear;
}
.caja1:hover{
  -webkit-transform: scale(.5);
  transform: scale(.5);
}
```

A continuación, se muestran algunos **ejemplos sencillos**:



```
<!DOCTYPE html>
<html>
<head>
<style>
div.a {
  width: 150px;
  height: 80px;
  background-color: orange;
  transform: rotate(20deg);
  transform-origin: bottom center; /*ejeX, ejeY*/
}

div.b {
  width: 150px;
  height: 80px;
  background-color: orange;
  transform: skew(30deg, 5deg);
  transform-origin: 50% -80%;
}

div.c {
  width: 150px;
  height: 80px;
  background-color: orange;
  transform: scaleY(1.5);
}

div.d {
  width: 150px;
  height: 80px;
  background-color: orange;
  transform: translateX(80px);
}
</style>
</head>
<body>

<h1>The transform Property</h1>

<h2>transform: rotate(20deg):</h2>
<div class="a">Hello World!</div>
<br>

<h2>transform: skew(30deg, 5deg):</h2>
<div class="b">Hello World!</div>
<br>

<h2>transform: scaleY(1.5):</h2>
<div class="c">Hello World!</div>

<h2>transform: translateX(80px):</h2>
<div class="d">Hello World!</div>

</body>
</html>
```

The transform Property

transform: rotate(20deg):



transform: skew(30deg, 5deg):



transform: scaleY(1.5):



transform: translateX(80px):





9. Transition

Las transiciones CSS permiten **cambiar** los valores de las propiedades **suavemente**, **durante una duración determinada**. Nos permiten **rotar, torcer, escalar o desplazar** los elementos de nuestra página web.

Este tipo de propiedades son muy interesantes para convertir el lenguaje de hojas de estilo en un sistema capaz de realizar todo tipo de efectos visuales.

Para crear un efecto de transición, se deben especificar **dos** cosas:

- 1) La **propiedad CSS** a la que se quiere añadir un efecto. **El efecto de la transición comenzará cuando la propiedad CSS especificada cambie de valor**.
- 2) La **duración** del efecto. Si no se especifica la duración, la transición no tendrá efecto, porque el valor por defecto es 0.

Para realizar las transiciones tenemos a nuestra disposición las siguientes **propiedades**:

- **transition-property**. Indica a qué propiedades queremos que se le aplique la transición. Por ejemplo, *all*, *width*, *height*, etc. Su **sintaxis** es la siguiente:

```
transition-property: none | all | property | initial | inherit;
```

- **transition-duration**. Indica la **duración de la transición**, es decir, cuánto tiempo queremos que tarde el navegador en reproducir el cambio entre las características iniciales a las características que le hemos indicado (no solo a características de *transform*, sino también a características como pueden ser el color).

```
transition-duration: time | initial | inherit;
```

- **transition-timing-function**. Especifica la **curva de velocidad** del efecto de transición. Permite que un efecto de transición **cambie de velocidad** a lo largo de su duración, por ejemplo, primero más rápido y luego más lento.

```
transition-timing-function: linear | ease | ease-in | ease-out |  
ease-in-out | step-start | step-end | steps(int,start|end)| cubic-  
bezier(n,n,n,n) | initial | inherit;
```




- *linear*: misma velocidad de principio a fin.
- *ease*: Valor por defecto. Especifica un inicio lento, luego rápido y un final lento.
- *ease-in*: comienzo lento.
- *ease-out*: final lento.
- *ease-in-out*: comienzo y final lentos.
- *steps(int, start/end)*: especifica una función de pasos. El primer parámetro especifica el número de **intervalos** de la función. Debe ser un número entero positivo (mayor que 0). El segundo parámetro, que es opcional, tiene el valor "start" o "end", y especifica el punto en el que se produce el cambio de valores dentro del intervalo. Si se omite el segundo parámetro, se le da el valor "end".
- *step-start*: equivalente a steps(1, start).
- *step-end*: equivalente a steps(1, end).
- *cubic-bezier(n,n,n,n)*: es una función de tiempo personalizada y por tanto podemos definir nuestros propios valores en la función cubic-bezier. Los valores posibles son valores numéricos de 0 a 1.
https://www.w3schools.com/cssref/tryit.php?filename=trycss_func_cubic-bezier

- *transition-delay*. Especifica el **retardo** (en segundos) para el efecto de transición.

```
transition-delay: time | initial | inherit;
```

- *transition*. Establece las **cuatro propiedades de transición anteriores en una sola propiedad**.

```
transition: transition-property | transition-duration | transition-timing-function | transition-delay | initial | inherit;
```

A continuación, se muestran un **ejemplo**:



```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background: red;
  transition-property: all;
  transition-duration: 5s;
  transition-timing-function: ease-in-out;
  transition-delay: 2s;
}
div:hover {
  transform: rotate(240deg) skew(20deg);
  background: green;
  width: 300px;
}
</style>
</head>
<body>

<h1>La propiedad transition</h1>
<p>Al poner el ratón sobre el elemento, este rotará 240º
y se deformará 20º. Al mismo tiempo se pondrá verde y
aumentará su ancho a 300px:</p>
<div></div>

</body>
</html>
```

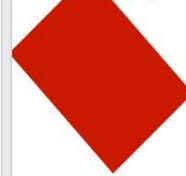
La propiedad transition

Al poner el ratón sobre el elemento, este rotará 240º y se deformará 20º. Al mismo tiempo se pondrá verde y aumentará su ancho a 300px:



La propiedad transition

Al poner el ratón sobre el elemento, este rotará 240º y se deformará 20º. Al mismo tiempo se pondrá verde y aumentará su ancho a 300px:



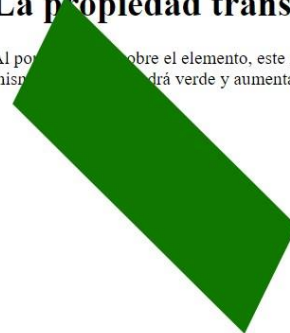
La propiedad transition

Al poner el ratón sobre el elemento, este rotará 240º y se deformará 20º. Al mismo tiempo se pondrá verde y aumentará su ancho a 300px:



La propiedad transition

Al poner el ratón sobre el elemento, este rotará 240º y se deformará 20º. Al mismo tiempo se pondrá verde y aumentará su ancho a 300px:





10. Iconografía

Los iconos pueden añadirse fácilmente a nuestro código HTML, utilizando una biblioteca de iconos como **Yesicon.app**, **Fontawesome**, **Google Icons**, **Bootstrap Icons**, o **icomoon**. Estas librerías son **vectores escalables** que se pueden personalizar con CSS (tamaño, color, sombra, etc.).

10.1 Font Awesome

Para utilizar los iconos gratuitos de Font Awesome, podemos optar por los siguientes dos métodos:



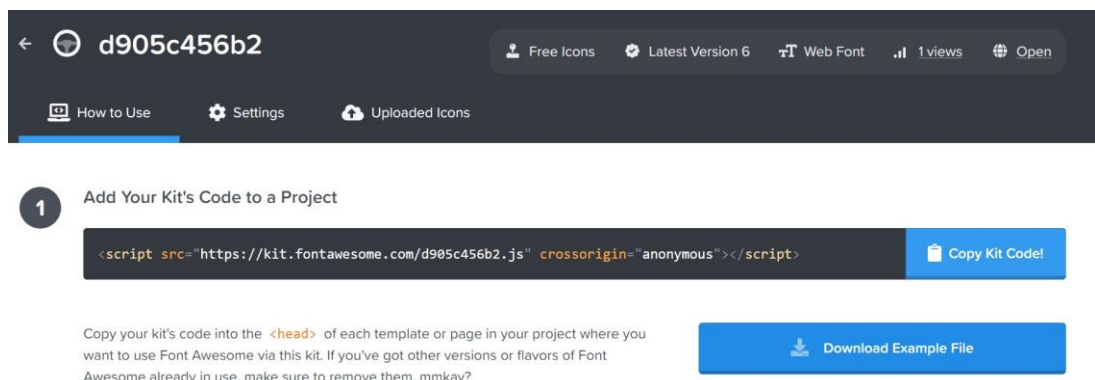
- 1) **Registrarnos** para obtener una cuenta en Font Awesome, y obtener un código (llamado KIT CODE) para utilizarlo cuando añadamos Font Awesome a nuestra página web.
- 2) **Descargar** la biblioteca de Font Awesome en nuestro almacenamiento local.

A continuación, se describe cómo añadir iconos y su aplicación de estilos por cada uno de los métodos mencionados.

10.1.1 Registro u online

Para integrar los iconos deseados en nuestro código HTML mediante el registro en la página u online, debemos llevar a cabo los siguientes pasos:

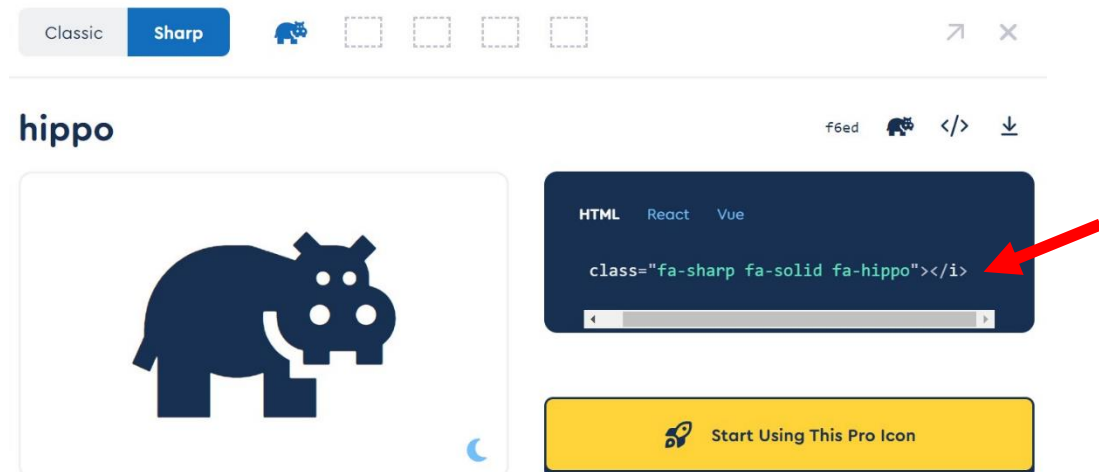
- 1) Nos registramos en la página de Font Awesome. Vamos a nuestro perfil, y allí encontraremos nuestro KIT Code, el cual insertaremos dentro del `<head>` de nuestro código HTML.





```
<!doctype html>
<html>
  <head>
    <!-- Place your kit's code here -->
    <script src="https://kit.fontawesome.com/d905c456b2.js" crossorigin="anonymous"></script>
  </head>
```

- 2) Navegamos en la página hasta encontrar los iconos que nos interesen, y **copiamos su código** en nuestro código HTML como cualquier otra clase, y le damos estilo.

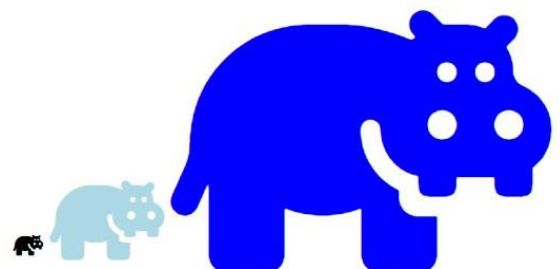


Font Awesome está diseñado para usar con elementos inline. Los elementos `<i>` y `<span>` son ampliamente utilizados para iconos.

Ejemplo:

```
<!doctype html>
<html>
  <head>
    <!-- Place your kit's code here -->
    <script src="https://kit.fontawesome.com/d905c456b2.js" crossorigin="anonymous"></script>
  </head>

  <body>
    <i class="fa-sharp fa-solid fa-hippo"></i>
    <i class="fa-sharp fa-solid fa-hippo" style="font-size:60px;color:lightblue;"></i>
    <i class="fa-sharp fa-solid fa-hippo" style="font-size:200px;color:blue;"></i>
  </body>
</html>
```



10.1.2 Descarga o local

Para integrar los iconos deseados en nuestro código HTML mediante su descarga en nuestro almacenamiento local, debemos llevar a cabo los siguientes pasos:

- 1) Descargamos los iconos de la página web (<https://fontawesome.com/download>) y los guardamos en una carpeta debidamente nombrada, por ejemplo, con el nombre de *icons*.
- 2) Insertaremos dentro del `<head>` de nuestro código HTML, el siguiente `<script>` con la ruta en donde hayamos guardado los iconos.

```
<!DOCTYPE html>
<html>
<head>
<title>Font Awesome Icons</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<script src="fonts/js/all.js"></script>
</head>
<body>

<h1>Font Awesome icon library</h1>

<p>Some Font Awesome icons:</p>
<span class="fas fa-cloud"></span>
<i class="fas fa-heart"></i>
<i class="fas fa-car"></i>
<i class="fas fa-file"></i>
<i class="fas fa-bars"></i>
<i class="fa-solid fa-envelope"></i>
<i class="fas fa-spinner fa-pulse"></i>

<p>Styled Font Awesome icons (size and color):</p>
<i class="fa-solid fa-envelope" style="font-size:24px;color:red;"></i>
<i class="fas fa-file" style="font-size:40px;"></i>
<span class="fas fa-cloud" style="font-size:36px;color:aqua;"></span>
<i class="fas fa-bars" style="font-size:48px;color:orange;"></i>
<i class="fas fa-heart" style="font-size:60px;color:lightblue;"></i>
<i class="fas fa-spinner fa-pulse" style="font-size:50px;color:red;"></i>

</body>
</html>
```

Este icono es animado.

La clase `fa-spin` consigue que cualquier icono gire, y la clase `fa-pulse` consigue que cualquier icono gire con 8 pasos.





Font Awesome icon library

Some Font Awesome icons:



Styled Font Awesome icons (size and color):



Toda la información relativa a los iconos de Fontawesome se pueden consultar en https://www.w3schools.com/icons/fontawesome5_intro.asp

10.2 Google Icons

Para utilizar los iconos de Google Icons, podemos optar por los siguientes dos métodos:

- 1) **Utilizarlos online, o a nivel externo**, cuando no queremos tener iconos en local. Esta opción es mucho más sencilla y se puede utilizar si no tenemos reparos para usar recursos externos en nuestra página web.
- 2) A **nivel interno**, descargando la fuente, y mediante el uso de **@font-face**.

A continuación, se describe cómo añadir iconos y su aplicación de estilos por cada uno de los métodos mencionados.

10.2.1 Registro u online

El proceso para utilizar los iconos de Google Icons directamente en nuestro código, es el siguiente:

- 1) En primer lugar, debemos insertar dentro del `<head>` de nuestro código HTML, la siguiente línea:

```
<link rel="stylesheet"
href="https://fonts.googleapis.com/icon?family=Material+Icons">
```




- 2) Se declarará una **clase** de **CSS** adicional llamada **.material-icons**. Cualquier elemento que use esta clase tendrá el CSS correcto para renderizar estos íconos desde la fuente web.

Un ejemplo sería el siguiente:

```
<!DOCTYPE html>
<html>
<head>
<title>Google Icons</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" href="https://fonts.googleapis.com/icon?family=Material+Icons">
</head>
<body>

<h1>Google icon library</h1>

<p>Some Google icons:</p>
<i class="material-icons">cloud</i>
<i class="material-icons">favorite</i>
<i class="material-icons">attachment</i>
<i class="material-icons">computer</i>
<i class="material-icons">traffic</i>
<span class="material-icons">logout</span>
<br><br>

<p>Styled Google icons (size and color):</p>
<i class="material-icons" style="font-size:24px;">cloud</i>
<i class="material-icons" style="font-size:36px;">favorite</i>
<i class="material-icons" style="font-size:48px;color:red;">attachment</i>
<i class="material-icons" style="font-size:60px;color:lightblue;">traffic</i>

</body>
</html>
```



Google icon library

Some Google icons:



Styled Google icons (size and color):





10.2.2 Descarga o local

Para aquellos desarrolladores web que quieran alojar la fuente web por su cuenta, se necesita un poco de configuración adicional.

- 1) Alojamos la fuente del icono que deseamos utilizar, en una ubicación (por ejemplo, <https://example.com/material-icons.woff>) y agregamos la siguiente regla de CSS:

```
@font-face {  
  font-family: 'Material Icons';  
  font-style: normal;  
  font-weight: 400;  
  src: url(https://example.com/MaterialIcons-Regular.eot); /* For IE6-8 */  
  src: local('Material Icons'),  
    local('MaterialIcons-Regular'),  
    url(https://example.com/MaterialIcons-Regular.woff2) format('woff2'),  
    url(https://example.com/MaterialIcons-Regular.woff) format('woff'),  
    url(https://example.com/MaterialIcons-Regular.ttf) format('truetype');  
}
```

- 2) Declaramos las reglas de CSS para renderizar el ícono a fin de renderizar la fuente de forma adecuada. A continuación, se muestra un ejemplo:

```
.material-icons {  
  font-family: 'Material Icons';  
  font-weight: normal;  
  font-style: normal;  
  font-size: 24px; /* Preferred icon size */  
  display: inline-block;  
  line-height: 1;  
  text-transform: none;  
  letter-spacing: normal;  
  word-wrap: normal;  
  white-space: nowrap;  
  direction: ltr;  
}
```

- 3) Añadimos los iconos. En el siguiente ejemplo se añade el icono de un rostro:

`<span class="material-icons">face</span>`



Podemos encontrar más información en las siguientes url:

https://developers.google.com/fonts/docs/material_icons#icon_font_for_the_web

https://developers.google.com/fonts/docs/material_icons



10.3 Icomoon

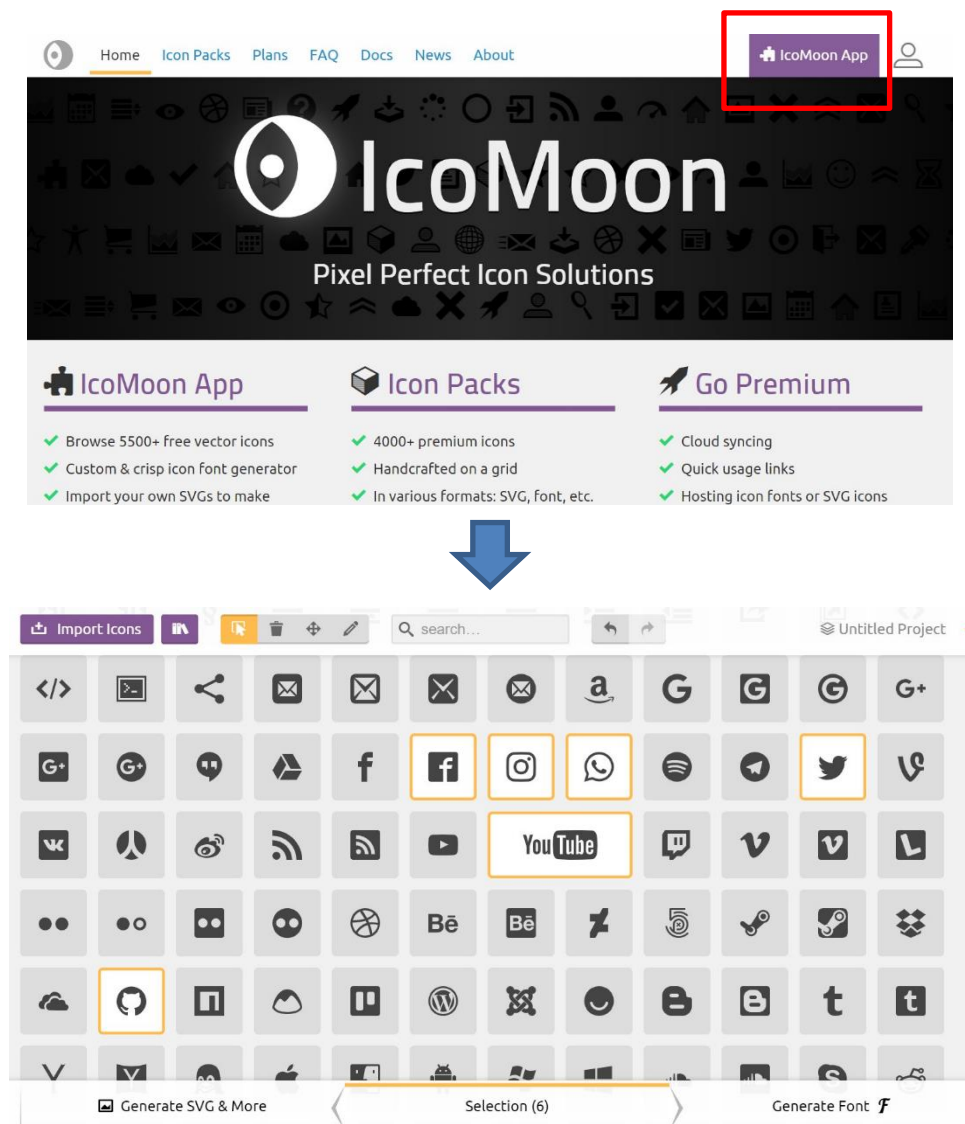
Para utilizar los iconos de Icomoon, podemos optar por los siguientes dos métodos:

- 1) **Utilizarlos online, o a nivel externo**, cuando no queremos tener iconos en local. Si no tenemos la cuenta premium, nos darán un acceso temporal que durará 24 horas.
- 2) A **nivel interno**, descargando la fuente en el almacenamiento local o hosting.

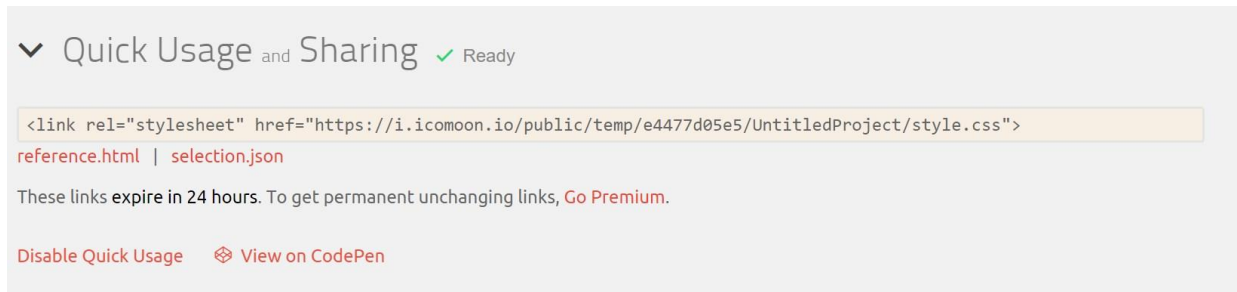
10.3.1 Registro u online

El **proceso** es el siguiente:

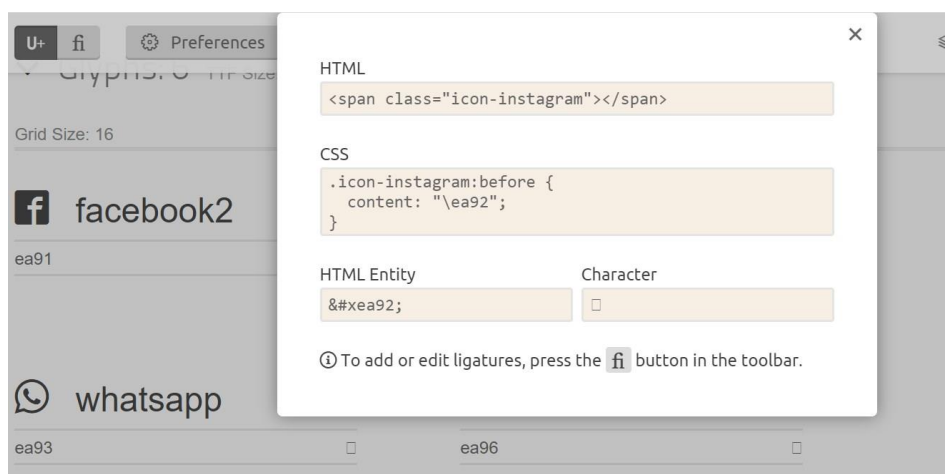
- 1) Vamos a la página web <https://icomoon.io/> y seleccionamos “IcoMoon App”. Al hacerlo, podremos **seleccionar** los iconos que nos interesen, y al finalizar clicamos sobre la opción “Generate Font”.



- 2) Seleccionamos la opción **“Quick Usage”**, y nos aparece un código `<link>` que introduciremos en el `<head>` de nuestro código HTML. Hay que tener especial cuidado ya que este link deja de estar disponible pasadas **24 horas**.



- 3) Añadimos cada uno de los iconos seleccionados, mediante el código HTML o CSS que se nos indica en la opción **“Get Code”** en cada uno de los respectivos iconos.



A continuación, se muestra un ejemplo.

```
<!DOCTYPE html>
<html>
<head>
<title>IcoMoon Icons</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" href="https://i.icomoon.io/public/temp/e4477d05e5/UntitledProject/style.css">
</head>
<body>

<p>Styled Icomoon icons (size and color):</p>
<span class="icon-youtube2" style="font-size:50px; color:red;"></span>
<span class="icon-facebook2" style="font-size:50px; color:blue;"></span>
<span class="icon-whatsapp" style="font-size:50px; color:green;"></span>
<span class="icon-instagram" style="font-size:60px; color:pink;"></span>
<span class="icon-twitter" style="font-size:60px; color:aqua;"></span>
</body>
</html>
```



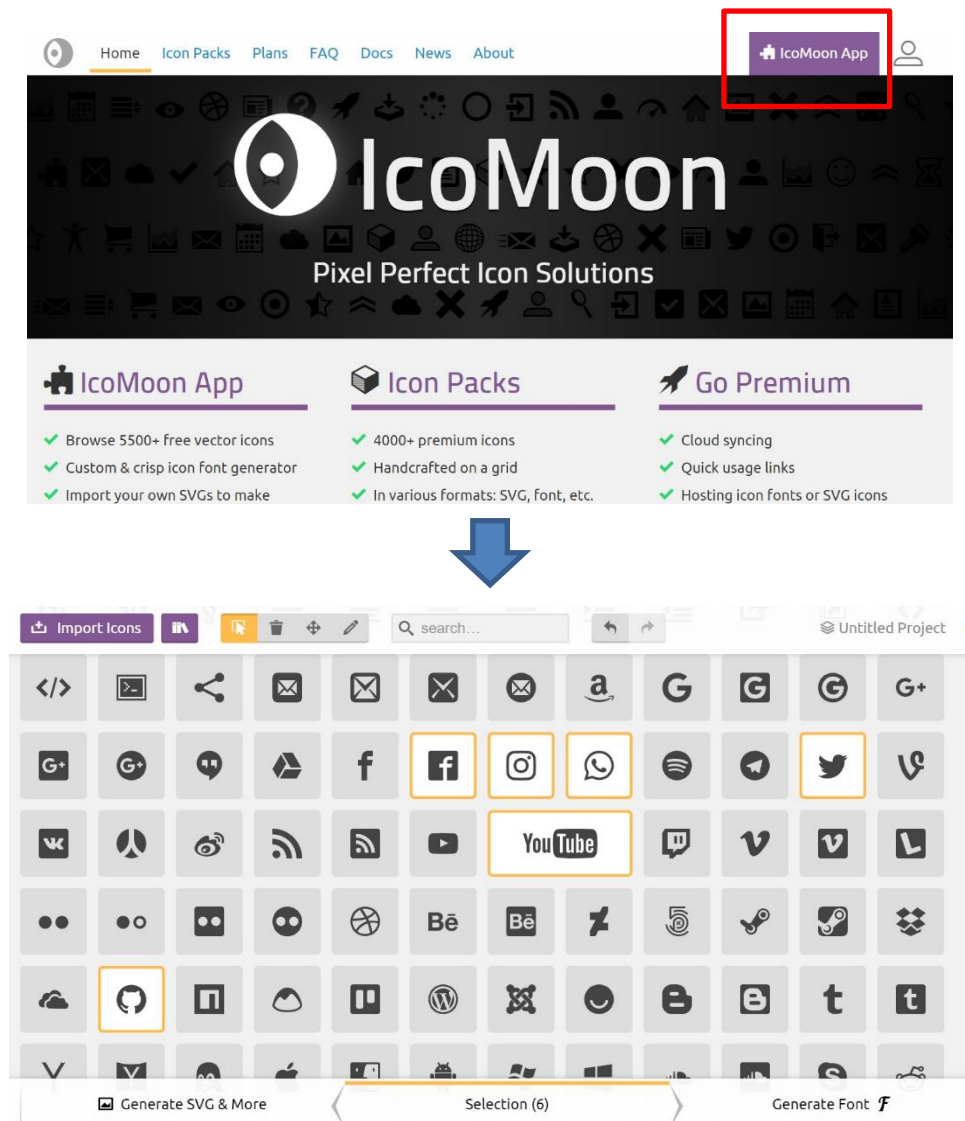
Styled Icomoon icons (size and color):



10.3.2 Descarga o local

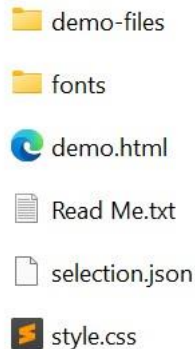
Debido a que el link que nos da icomoon sólo está disponible 24 horas, nos conviene descargar y almacenar los iconos en nuestro propio hosting o ficheros locales de nuestra página web. En este caso, para utilizar dichos iconos, debemos seguir los pasos que se indican a continuación.

- 1) Vamos a la página web <https://icomoon.io/> y seleccionamos “IcoMoon App”. Al hacerlo, podremos **seleccionar** los iconos que nos interesen, y al finalizar clicamos sobre la opción “**Generate Font**”.





- 2) Seleccionamos en la opción "Download", y nos descargará un fichero .zip que descomprimiremos. El archivo descargado contendrá el siguiente contenido:



- 3) En la carpeta /fonts tendremos nuestros iconos, que copiaremos a nuestra carpeta /fonts de nuestro proyecto. Además, icoMoon ya nos da el código CSS hecho, que solo tendremos que vincular a nuestro código HTML.

A continuación, se muestra un ejemplo:

```
<!DOCTYPE html>
<html>
<head>
<title>IcoMoon Icons</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" href="style_icomoon.css">
</head>
<body>

<p>Styled Icomoon icons (size and color):</p>
<span class="icon-youtube2" style="font-size:50px; color:red;"></span>
<span class="icon-facebook2" style="font-size:50px; color:blue;"></span>
<span class="icon-whatsapp" style="font-size:50px; color:green;"></span>
<span class="icon-instagram" style="font-size:60px; color:pink;"></span>
<span class="icon-twitter" style="font-size:60px; color:aqua;"></span>
<span class="icon-github" style="font-size:60px;"></span>
</body>
</html>
```



Styled Icomoon icons (size and color):





11. Imágenes SVG en HTML y CSS

SVG es un **formato de archivo basado en XML** que describe gráficos vectoriales de forma bidimensional. A diferencia de las imágenes rasterizadas (como JPEG o PNG), los SVG son escalables, lo que significa que conservan la calidad y nitidez independientemente del tamaño al que se escalen.

11.1 Cómo insertar imágenes SVG

Existen varias **formas** de **insertar** imágenes SVG en un documento HTML.

11.1.1 Usando la etiqueta `<img>` para un SVG

Para agregar una imagen SVG a tu página utilizando la etiqueta `<img>`, simplemente especifica la ruta del archivo SVG en el atributo `src` y proporciona un texto alternativo en el atributo `alt` para accesibilidad. Ejemplo:

```

```

Cuando se añade una imagen SVG mediante la etiqueta `<img>` sin especificar el tamaño, se utiliza el tamaño original del archivo SVG. Para cambiar este tamaño, es necesario especificar las dimensiones utilizando CSS o HTML. También es posible modificar las dimensiones originales del SVG directamente, esto lo veremos más adelante. Veamos algunos ejemplos de cómo cambiar el tamaño con CSS y HTML:

1) Ejemplo de cambio de tamaño de SVG con CSS:

Si queremos ajustar el tamaño de un SVG utilizando CSS, podemos hacerlo de la siguiente manera:

```
<!-- Imagen SVG sin especificar tamaño -->

img.svg-img {
    width: 200px;
    height: auto; /* Mantener La proporción de aspecto */
}
```



2) Ejemplo de cambio de tamaño de SVG en HTML:

También puedes indicar el tamaño directamente en la etiqueta `<img>` utilizando los atributos `width` y `height`.

```

```

11.1.2 Usando SVG en línea (inline)

Las imágenes SVG pueden escribirse directamente en el documento HTML mediante la etiqueta `<svg>` `</svg>`. Sigue estos pasos para incorporar un SVG inline en tu página:

1. Abre el archivo SVG en tu editor de código (como VS Code).
2. Copia todo el código SVG dentro de las etiquetas `<svg>` `</svg>`.
3. Pega el código SVG dentro del elemento `<body>` en tu documento HTML.

```
<body>
<!-- Pega aquí el código SVG -->
<svg xmlns="http://www.w3.org/2000/svg" width="100" height="100"
viewBox="0 0 100 100">
<circle cx="50" cy="50" r="40" stroke="black" stroke-width="3"
fill="red" />
</svg>
</body>
```

Cuando se utiliza SVG inline en el documento HTML, se reduce el tiempo de carga porque el SVG se sirve como parte del HTML principal, evitando una solicitud HTTP adicional. Sin embargo, el código HTML se hace más extenso y engorroso.

Esta técnica **nos permite realizar una mayor personalización** en comparación con el uso de `<img>`.



11.1.3 Usando la etiqueta <object> para un SVG

Otra forma de insertar imágenes SVG en un documento HTML es mediante la etiqueta <object>. Esta etiqueta nos permite incrustar un archivo SVG como un objeto independiente en la página.

Puedes utilizar la etiqueta <object> para insertar un archivo SVG y definir el tamaño del objeto utilizando CSS o atributos HTML `width` y `height`.

```
<object type="image/svg+xml" data="imagen.svg" width="300" height="200">
```

Tu navegador no soporta SVGs.

```
</object>
```

En este ejemplo:

- El atributo `type="image/svg+xml"` especifica el tipo de archivo como SVG.
- El atributo `data="imagen.svg"` indica la ruta del archivo SVG que se debe mostrar.
- Los atributos `width="300"` y `height="200"` definen las dimensiones del objeto SVG en la página.

La etiqueta <object> ofrece flexibilidad al incrustar SVGs, permitiendo un control preciso sobre el tamaño y la presentación del gráfico vectorial en tu página web.

11.1.4 Usando la etiqueta <iframe> con SVG

Otra técnica para insertar imágenes SVG en un documento HTML es mediante el uso de la etiqueta <iframe>. Sin embargo, es importante tener en cuenta ciertas consideraciones al utilizar <iframe> para mostrar SVGs.

Puedes utilizar la etiqueta <iframe> para incrustar un archivo SVG en tu página web:

```
<iframe src="imagen.svg" width="300" height="200" title="SVG"></iframe>
```

En este ejemplo:



- El atributo `src="imagen.svg"` especifica la URL del archivo SVG que se debe mostrar dentro del `<iframe>`.
- Los atributos `width="300"` y `height="200"` definen las dimensiones del `<iframe>` para mostrar el SVG.
- El atributo `title="SVG"` proporciona un título descriptivo para el `<iframe>`.

Aunque `<iframe>` puede ser una forma rápida de mostrar SVGs en tu página, existen algunas **consideraciones** importantes:

- 1) **Dificultades de mantenimiento.** Los `<iframe>` pueden ser difíciles de mantener, especialmente cuando tienes múltiples elementos `<iframe>` en tu página.
- 2) **SEO (Optimización para Motores de Búsqueda).** El uso excesivo de `<iframe>` puede afectar negativamente la optimización de tu sitio para motores de búsqueda.
- 3) **Escalabilidad de SVG.** Las imágenes SVG añadidas mediante `<iframe>` no son escalables, lo que anula el propósito de utilizar gráficos vectoriales escalables.

11.1.5 Usando la propiedad `background-image` con SVG

Otra forma de incorporar imágenes SVG en un diseño web es utilizando la propiedad `background-image` en CSS. Esta técnica nos permite **aplicar un SVG como fondo** de un elemento HTML, como un `<div>`.

Puedes utilizar `background-image` en tu hoja de estilos CSS para aplicar un SVG como fondo de un elemento HTML:

```
<div class="svg-background">  
  <!-- Contenido dentro del div con imagen SVG de fondo -->  
</div>
```




```
.svg-background {  
width: 300px;  
height: 200px;  
background-image: url('imagen.svg');  
background-size: cover;  
background-repeat: no-repeat; }
```

En este ejemplo:

- La clase `.svg-background` se utiliza para aplicar estilos al `<div>` que contendrá la imagen SVG como fondo.
- La propiedad `background-image` especifica la URL del archivo SVG que se utilizará como fondo.
- `background-size: cover`; asegura que la imagen SVG se ajuste completamente al tamaño del `<div>`.
- `background-repeat: no-repeat`; evita que la imagen SVG se repita en el fondo del `<div>`.

Aunque `background-image` es una forma conveniente de aplicar SVGs como fondos en HTML, es importante tener en cuenta lo siguiente:

- **No Accesible.** Las imágenes de fondo SVG no son accesibles a través del texto alternativo (`alt`) como las imágenes `<img>`.
- **Escalabilidad.** La escalabilidad del SVG puede verse limitada dependiendo del contenedor y las propiedades CSS aplicadas.



11.2 Cambiar las propiedades de un SVG con CSS

Los archivos **SVG** (Scalable Vector Graphics) pueden ser fácilmente manipulados y estilizados utilizando CSS. Esto te permite cambiar aspectos como color, tamaño, posición y animación de los elementos dentro del SVG.

La manipulación de las propiedades CSS de los SVG funciona bien con SVG simples, como círculos o formas básicas, pero puede encontrarse con dificultades al aplicarse a SVG más complejos debido a su estructura anidada y la presencia de múltiples elementos dentro del SVG. Por ejemplo, en SVGs complejos que contienen grupos `<g>`, rutas `<path>`, y otros elementos, la aplicación de estilos CSS puede no afectar al elemento deseado directamente debido a la jerarquía y la especificidad de los selectores.

11.2.1 Cambiar el color de una imagen SVG

Podemos cambiar el color de un SVG utilizando la propiedad `fill` en CSS. Esto es útil para personalizar los colores de partes específicas de un gráfico vectorial.

```
/* Cambiar el color de todos los elementos del SVG a rojo */  
svg {  
  fill: red;  
}  
/* Cambiar el color de un elemento específico por su clase */  
.circulo {  
  fill: blue;  
}
```

11.2.2 Ajustar el tamaño de una imagen SVG

Podemos ajustar el tamaño de un SVG utilizando las propiedades `width` y `height` en CSS. Esto te permite controlar las dimensiones del SVG en relación con su contenedor.

```
/* Ajustar el tamaño del SVG a 200px de ancho */  
svg {  
  width: 200px;  
  height: auto; /* Mantener la proporción de aspecto */  
}
```

A continuación, se presenta un ejemplo en CodePen de cómo se puede manipular un SVG simple con CSS:

HTML

```

1 <svg viewBox="0 0 100 100">
2 <!-- Definición de un círculo con centro en (50,50)
   y radio de 40 -->
3 <circle cx="50" cy="50" r="40" fill="#4682b4" />
4 </svg>

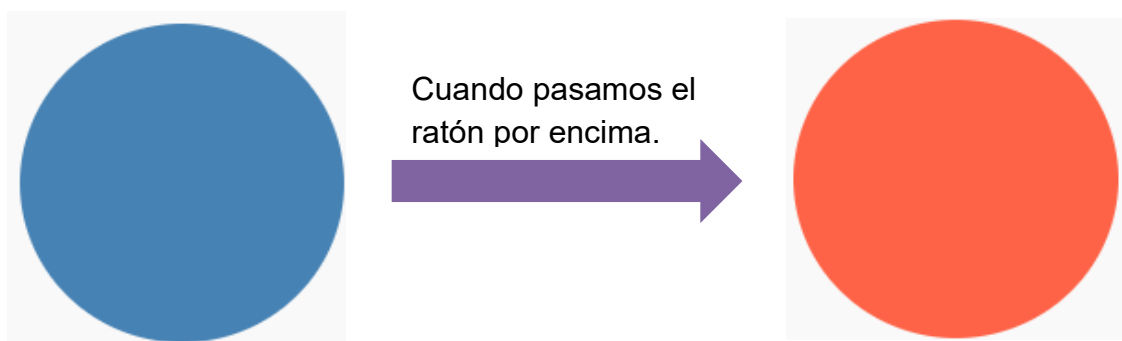
```

CSS

```

1 body {
2   display: flex;
3   justify-content: center;
4   align-items: center;
5   height: 100vh;
6   margin: 0;
7   background-color: #f9f9f9;
8 }
9
10 svg {
11   width: 200px;
12   height: 200px;
13   transition: fill 0.3s ease; /* Transición suave del color */
14   cursor: pointer;
15 }
16
17 /* Cambiar el color del círculo al pasar el mouse */
18 svg:hover circle {
19   fill: #ff6347; /* Nuevo color al pasar el mouse */
20 }

```



Además de cambiar el color y el tamaño, los SVG también permiten modificar otras propiedades visuales mediante CSS. Por ejemplo, podemos ajustar la opacidad de un SVG utilizando la propiedad `opacity`, podemos aplicar **animaciones**, cambiar su posición o usar **transiciones** y **transformaciones**, **filtros**, entre otras cosas.

11.2.3 Cambiar la opacidad de una imagen SVG

Podemos cambiar la opacidad de un SVG utilizando la propiedad `opacity` en CSS. Esto permite hacer que el SVG sea más transparente o más opaco.

```

/* Hacer que el SVG sea semi-transparente */

svg {
  opacity: 0.5; }

```



11.2.4 Cambiar la posición de una imagen SVG

Podemos cambiar la posición de un SVG utilizando las propiedades `position`, `top`, `left`, `right` y `bottom` en CSS. Esto te permite mover el SVG dentro de su contenedor.

```
/* Posicionar el SVG en el centro de su contenedor */  
  
svg {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%);  
}
```

11.2.5 Animar una imagen SVG con CSS

Podemos animar los elementos dentro de un SVG utilizando las animaciones CSS. Esto permite crear efectos visuales dinámicos y atractivos.

```
/* Definir una animación para cambiar el color de un elemento */  
  
@keyframes cambiarColor {  
  0% { fill: red; }  
  50% { fill: blue; }  
  100% { fill: green; }  
}  
  
/* Aplicar la animación al elemento del SVG */  
  
.circulo {  
  animation: cambiarColor 2s infinite alternate;  
}
```



12. Filtros CSS para aplicar efectos en imágenes

Los filtros CSS permiten aplicar efectos visuales a las imágenes. Para aplicar un filtro CSS a una imagen se utiliza la propiedad *filter*. Su **sintaxis** es la siguiente:

```
filter: none | blur() | brightness() | contrast() | drop-shadow() |  
grayscale() | hue-rotate() | invert() | opacity() | saturate() |  
sepia() | url();
```

Filtro	Descripción
none	Es el valor por defecto. No provoca ningún efecto.
blur	Aplica un efecto de desenfoque a la imagen. Un valor mayor creará más desenfoque. Si no se especifica ningún valor, se utiliza 0.
brightness	Ajusta el brillo de la imagen. 0% hará que la imagen sea completamente negra. 100% (1) es el valor por defecto y representa la imagen original. Valores superiores a 100% proporcionarán resultados más brillantes.
contrast	Ajusta el contraste de la imagen. 0% hará que la imagen sea completamente negra. 100% (1) es el valor por defecto, y representa la imagen original. Valores superiores a 100% proporcionarán resultados con más contraste.
drop-shadow	Aplica un efecto de sombra paralela a la imagen. Es similar a la propiedad box-shadow . Posibles valores: h-shadow - Obligatorio. Especifica un valor en píxeles para la sombra horizontal. Los valores negativos colocan la sombra a la izquierda de la imagen. v-shadow - Obligatorio. Especifica un valor en píxeles para la sombra vertical. Los valores negativos sitúan la sombra por encima de la imagen. blur - Opcional. Este es el tercer valor, y debe estar en píxeles. Añade un efecto de desenfoque a la sombra. Un valor mayor creará más desenfoque (la sombra se hace más grande y más clara). No se permiten valores negativos. Si no se especifica ningún valor, se utiliza 0 (el borde de la sombra es nítido). spread - Opcional. Este es el cuarto valor, y debe estar en píxeles. Los valores positivos harán que la sombra se expanda y se haga más grande, y los valores negativos harán que la sombra se encoja. Si no se especifica, será 0 (la sombra tendrá el mismo tamaño que el elemento). Nota: Chrome, Safari y Opera, y quizás otros navegadores, no soportan esta 4ª longitud; color - Opcional. Añade un color a la sombra. Si no se especifica, el color depende del navegador (a menudo negro).



grayscale	Convierte la imagen a escala de grises . 0% (0) es el valor por defecto y representa la imagen original. 100% hará que la imagen sea completamente gris (se utiliza para imágenes en blanco y negro). Nota: No se permiten valores negativos.
hue-rotate	Aplica una rotación del tono a la imagen . El valor define el número de grados alrededor del círculo cromático que se ajustarán las muestras de la imagen. 0deg es el valor por defecto, y representa la imagen original. Nota: El valor máximo es 360deg.
invert	Invierte las muestras en la imagen. 0% (0) es el valor por defecto y representa la imagen original. 100% hará que la imagen esté completamente invertida. Nota: No se permiten valores negativos.
opacity	Establece el nivel de opacidad de la imagen. Este filtro es similar a la propiedad opacity . El nivel de opacidad describe el nivel de transparencia, donde: 0% es completamente transparente. 100% (1) es el valor por defecto y representa la imagen original (sin transparencia). Nota: No se permiten valores negativos.
saturate	Satura la imagen. 0% (0) hará que la imagen esté completamente desaturada. 100% es el valor por defecto y representa la imagen original. Valores superiores a 100% proporcionan resultados supersaturados. Nota: No se permiten valores negativos.
sepia	Convierte la imagen a sepia . 0% (0) es el valor por defecto y representa la imagen original. 100% hará que la imagen sea completamente sepia. Nota: No se permiten valores negativos.
url	La función url() toma la ubicación de un archivo XML que especifica un filtro SVG , y puede incluir un ancla a un elemento de filtro específico. Ejemplo: <i>filter: url(svg-url#element-id)</i>

A continuación, se muestra un **ejemplo** de cómo aplicar un **filtro de desenfoque** a una imagen:

HTML:

```

```

CSS:

```
.filtro-desenfoque {  
    filter: blur(5px);  
}
```



En este ejemplo, la clase filtro-desenfoco se aplica a la etiqueta `<img>` para seleccionar la imagen a la que se le aplicará el filtro. La propiedad `filter` se utiliza con el valor `blur(5px)` para aplicar un desenfoque de 5 píxeles a la imagen.

Se puede ajustar el filtro y su valor según el efecto que se desee lograr. También se pueden combinar múltiples filtros separándolos por espacios en la propiedad `filter`, por ejemplo: `filter: blur(5px) grayscale(100%);` aplicaría tanto un desenfoque como una escala de grises a la imagen.

Es importante tener en cuenta que **la compatibilidad de los filtros CSS puede variar entre los navegadores**.

12.1 Ejemplos de filtros en CSS

A continuación, se presentan algunos **ejemplos** de filtros CSS con una breve explicación. **Los valores utilizados en los ejemplos son solo ilustrativos** y se pueden ajustar según tus necesidades.

A. EJEMPLO 1. `filter: grayscale(100%);` Convierte la imagen a **escala de grises**.

HTML:

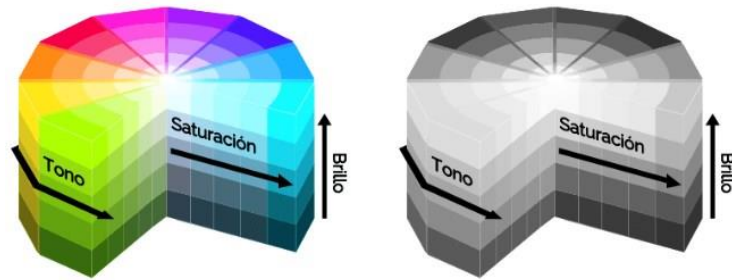
```



```

CSS:

```
.filtro{
  filter: grayscale(100%);
}
```



B. EJEMPLO 2. `filter: sepia(100%);` Aplica un **tono sepia** a la imagen.

HTML:

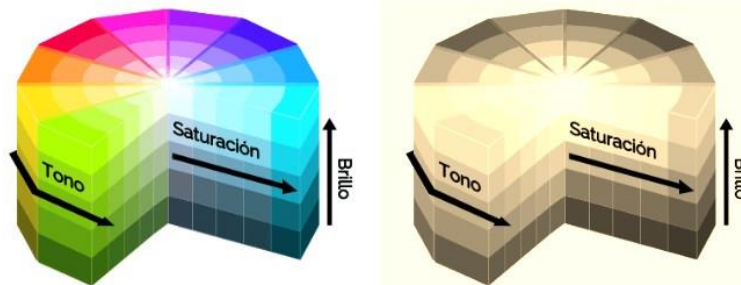
```



```

CSS:

```
.filtro{
  filter: sepia(100%);
}
```





C. EJEMPLO 3. *filter: blur(5px);* Aplica un efecto de **desenfoque** a la imagen.

HTML:

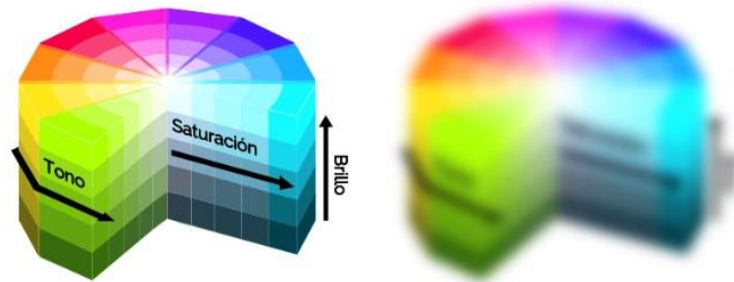
```



```

CSS:

```
.filtro{
  filter: blur(5px);
}
```



D. EJEMPLO 4. *filter: brightness(190%);* Ajusta el **brillo** de la imagen.

HTML:

```

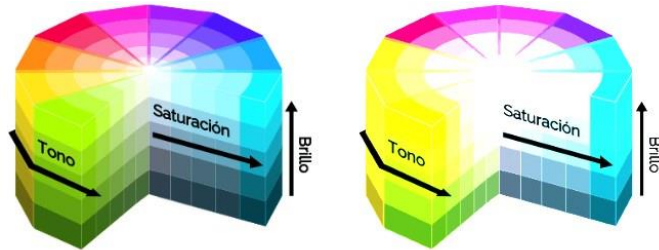


```



CSS:

```
.filtro{  
  filter: brightness(190%);  
}
```



E. **EJEMPLO 5.** *filter: contrast(200%);* Ajusta el **contraste** de la imagen.

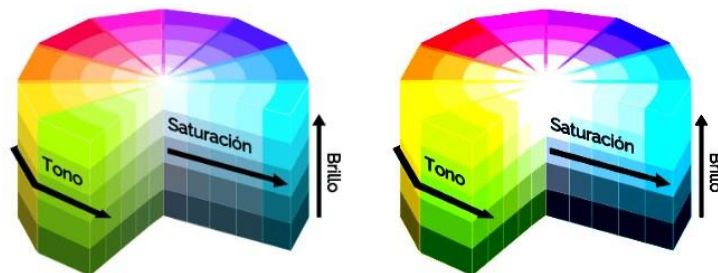
HTML:

```
  
  

```

CSS:

```
.filtro{  
  filter: contrast(200%);  
}
```





F. EJEMPLO 6. *filter: saturate(200%);* Aumenta o disminuye la **saturación** de la imagen.

HTML:

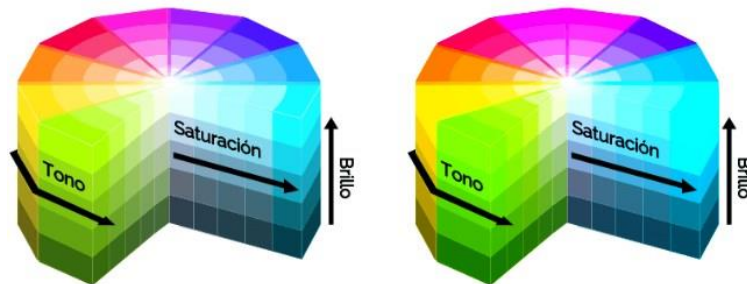
```



```

CSS:

```
.filtro{
  filter: saturate(200%);
}
```



G. EJEMPLO 7. *filter: hue-rotate(90deg);* Rota el matiz (tono) de la imagen.

HTML:

```

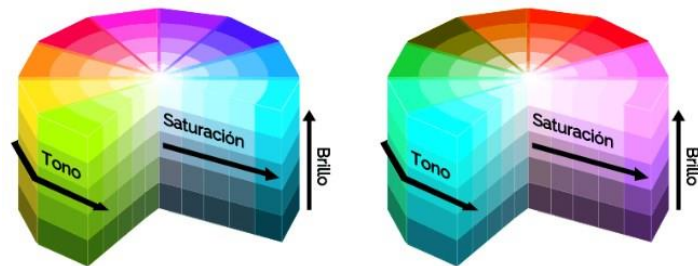


```



CSS:

```
.filtro{  
  filter: hue-rotate(90deg);  
}
```



H. **EJEMPLO 8.** *filter: invert(100%);* Invierte los colores de la imagen.

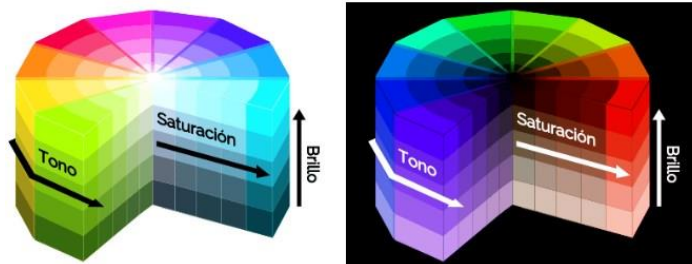
HTML:

```
  
  

```

CSS:

```
.filtro{  
  filter: invert(100%);  
}
```





I. EJEMPLO 9. *filter: opacity(50%);* Ajusta la opacidad de la imagen.

HTML:

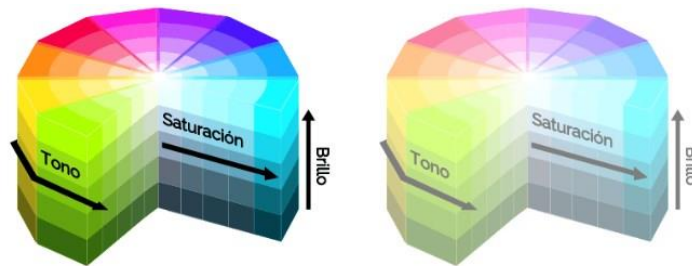
```



```

CSS:

```
.filtro{
  filter: opacity(50%);
}
```





13. Blend Modes CSS

Los **Blend Modes**, o **modos de mezcla** en español, son una característica de **CSS** que permite combinar elementos HTML con otros elementos de fondo como imágenes o degradados. Estos modos nos permiten **mezclar los colores del fondo, que puede ser una imagen o un color, con el elemento que se superpone**.

Los **Blend Modes en CSS** son una propiedad, de reciente incorporación, llamada **mix-blend-mode**, que permite **diferentes valores**, como por ejemplo para **oscurecer, aclarar o contrastar**, entre otros modos. Estos valores permiten mezclar el color de fondo de un elemento con el color del elemento superpuesto, que puede ser un texto o un contenedor.

Con las Blend Modes, **podemos jugar con la transparencia y la opacidad de los elementos**, logrando efectos de fusión, aclarado, superposición, saturación o contraste, entre otros.

13.1 Tipos de Blend Modes y sus efectos visuales

La siguiente tabla muestra los valores más destacados de la propiedad **mix-blend-mode**, su descripción y sus características.

Blend Mode	Descripción	Características
multiply	Multiplica los colores de la capa superior con los de la capa inferior.	Oscurece la imagen resultante.
screen	Invierte los colores de la capa superior y los multiplica con los de la capa inferior.	Aclara las áreas de la imagen.
overlay	Combina Multiply y Screen, oscureciendo o aclarando la capa inferior dependiendo de sus colores.	Produce un efecto de contraste y saturación en la imagen resultante.
darken	Conserva los colores más oscuros de las dos capas.	Selecciona el color más oscuro de cada píxel de ambas capas.
lighten	Conserva los colores más claros de las dos capas.	Selecciona el color más claro de cada píxel de ambas capas.
color-dodge	Aclara los colores de la capa inferior dependiendo de los colores de la capa superior.	Aclara las áreas de la imagen de la capa inferior.
color-burn	Oscurece los colores de la capa inferior dependiendo de los colores de la capa superior.	Oscurece las áreas de la imagen de la capa inferior.
difference	Resta los colores de la capa superior de los de la capa inferior.	Produce un efecto de inversión de colores.



exclusion	Similar a Difference, pero con una apariencia menos extrema.	Combina los colores de ambas capas con un efecto de exclusión.
hard-light	Combina Multiply y Screen, pero de una manera más intensa.	Produce un efecto más fuerte de contraste y saturación.
soft-light	Similar a Overlay, pero con una apariencia más suave.	Produce un efecto más suave de contraste y saturación.
hue	Preserva el brillo y la saturación de la capa inferior, pero adopta el tono de la capa superior.	Aplica el tono de la capa superior a la capa inferior.
saturation	Preserva el brillo y el tono de la capa inferior, pero adopta la saturación de la capa superior.	Aplica la saturación de la capa superior a la capa inferior.
color	Preserva el brillo de la capa inferior, pero adopta el tono y la saturación de la capa superior.	Aplica el tono y la saturación de la capa superior a la capa inferior.
luminosity	Preserva el tono y la saturación de la capa inferior, pero adopta el brillo de la capa superior.	Aplica el brillo de la capa superior a la capa inferior.

13.2 Implementación de Blend Modes en CSS

Para aplicar Blend Modes se utiliza la propiedad `mix-blend-mode`, que permite mezclar los colores de un elemento con los colores del fondo o de otros elementos superpuestos.

Su **sintaxis** es la siguiente:

```
.contenedor-superpuesto {  
    mix-blend-mode: <modo-de-mezcla>;  
}
```

Donde `<modo-de-mezcla>` es el valor del Blend Mode que quieres aplicar, como “`multiply`”, “`screen`”, “`overlay`”, entre otros.

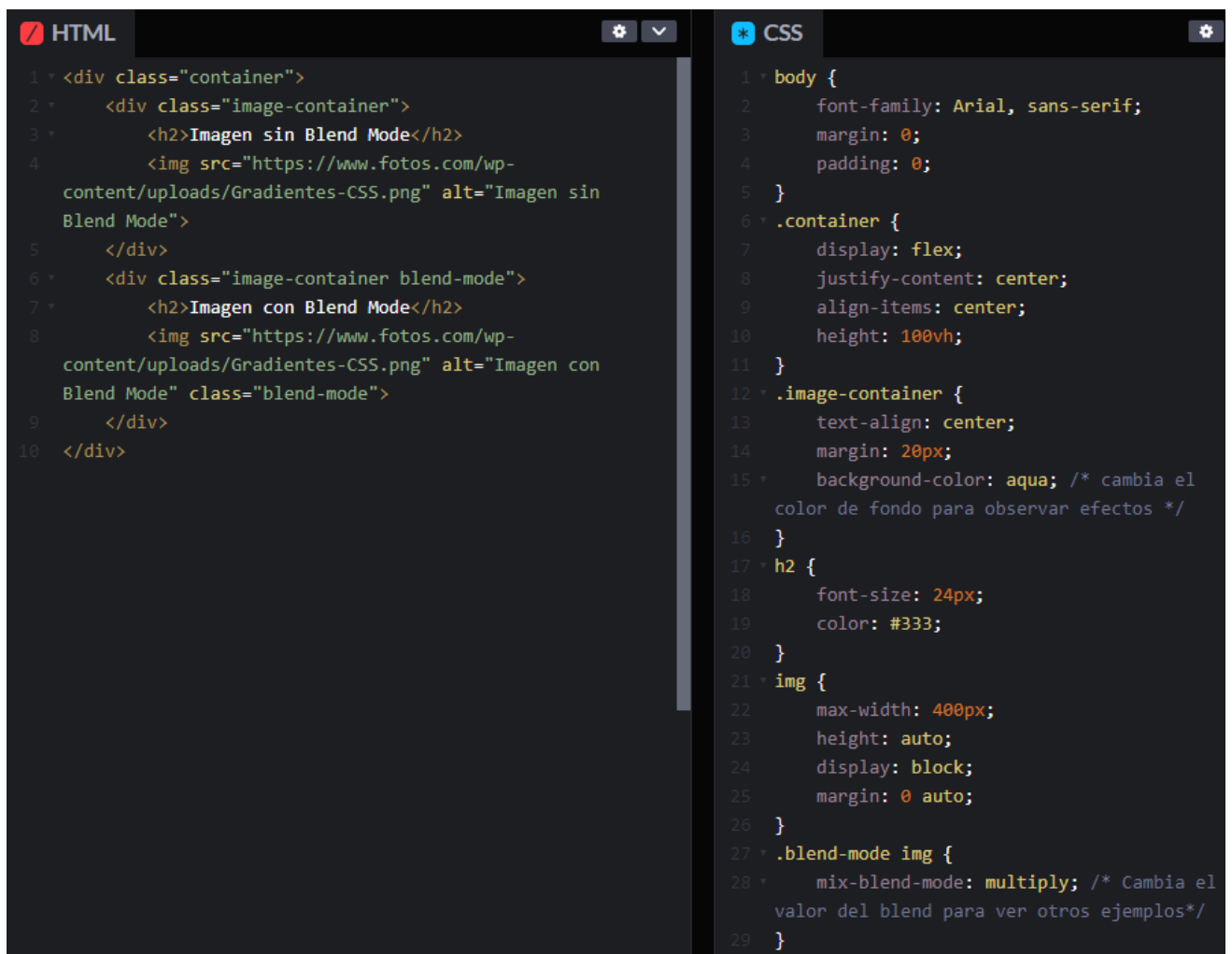
Por ejemplo, si queremos que un texto se mezcle con el fondo utilizando el modo “`multiply`”, crearíamos una regla CSS como esta:

```
.contenedor-superpuesto {  
    mix-blend-mode: multiply;  
}
```

Es esencial tener en cuenta que para que los Blend Modes funcionen correctamente, **necesitamos tener al menos dos elementos superpuestos**: uno con un fondo, como una imagen o un color de fondo, y otro con contenido superpuesto, como texto, contenedores o formas.

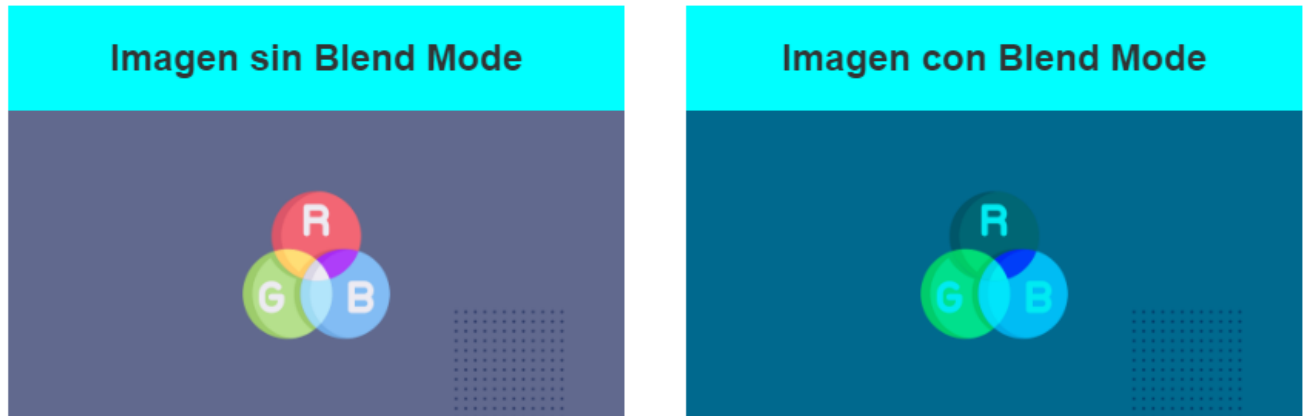
```
<div class="contenedor-fondo">
  <!-- Aquí colocaríamos nuestra imagen de fondo o un color de fondo -->
  <div class="contenedor-superpuesto">
    <p class="texto-superpuesto">Este texto se mezclará con el fondo
    utilizando el modo "multiply".</p>
  </div>
</div>
```

Es importante mencionar que la propiedad `mix-blend-mode` afecta únicamente a los **elementos secundarios y no al elemento padre**.



```
HTML
1 <div class="container">
2   <div class="image-container">
3     <h2>Imagen sin Blend Mode</h2>
4     
5   </div>
6   <div class="image-container blend-mode">
7     <h2>Imagen con Blend Mode</h2>
8     
9   </div>
10 </div>

CSS
1 body {
2   font-family: Arial, sans-serif;
3   margin: 0;
4   padding: 0;
5 }
6 .container {
7   display: flex;
8   justify-content: center;
9   align-items: center;
10  height: 100vh;
11 }
12 .image-container {
13   text-align: center;
14   margin: 20px;
15   background-color: aqua; /* cambia el color de fondo para observar efectos */
16 }
17 h2 {
18   font-size: 24px;
19   color: #333;
20 }
21 img {
22   max-width: 400px;
23   height: auto;
24   display: block;
25   margin: 0 auto;
26 }
27 .blend-mode img {
28   mix-blend-mode: multiply; /* Cambia el valor del blend para ver otros ejemplos */
29 }
```

13.3 Diferencias entre Blend Modes y Filtros CSS

Los Blend Modes en CSS y los **filtros CSS** son técnicas diferentes para manipular la apariencia de elementos en una página web. Mientras que los **Blend Modes controlan cómo se mezclan los colores de elementos superpuestos**, los **filtros CSS aplican efectos como desenfoque, saturación y cambios en el color de toda una imagen o elemento**. Por tanto:

- **Blend Modes:** mezcla entre dos elementos superpuestos.
- **Filtros:** efectos en una imagen o elemento.

Los Blend Modes son más útiles cuando se necesita mezclar dos o más elementos superpuestos, mientras que los filtros son más adecuados para aplicar efectos globales a elementos individuales, como imágenes.

13.4 Ejemplos prácticos de Blend Modes

Los Blend Modes ofrecen una amplia gama de posibilidades para crear efectos visuales en nuestro sitio web. Veamos algunos ejemplos prácticos de cómo podemos utilizarlos.

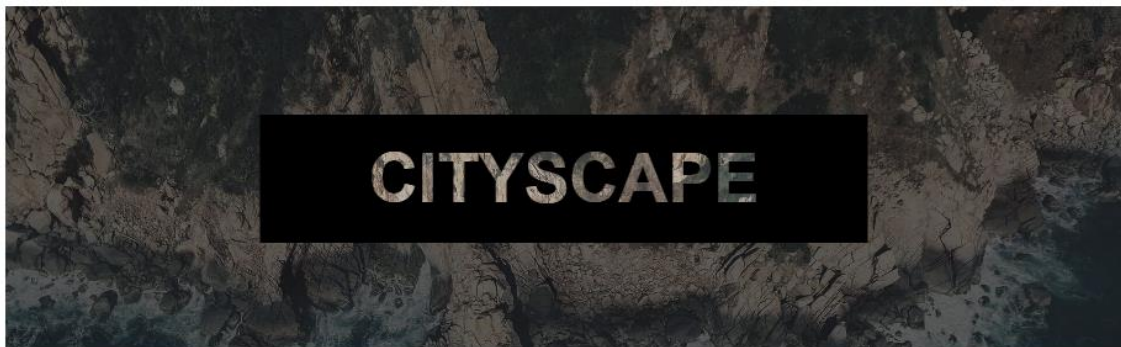
13.4.1 Efectos de texto creativos con overlay

En este ejemplo se aplica el modo de mezcla “**overlay**” a un texto superpuesto sobre imágenes de fondo para conseguir un efecto de superposición llamativo y legible.



```
HTML
1 <div class="image-container">
2   <div class="text">CITYSCAPE</div>
3 </div>
4 <h2>Modern Cityscape Text Effect</h2>
5 <p>This example creates a modern text effect
  with a cityscape background image.</p>
6 <p>Resize the browser window to see the
  responsive effect.</p>

CSS
1 body {font-family: Arial, Helvetica, sans-serif;}
2
3 .image-container {
4   background-image: url("https://www.fotos.com/wp-
  content/uploads/home-historia.jpg");
5   background-size: cover;
6   position: relative;
7   height: 280px;
8 }
9 .text {
10  background-color: black;
11  color: white;
12  font-size: 6vw;
13  font-weight: bold;
14  margin: 0 auto;
15  padding: 20px;
16  width: 50%;
17  text-align: center;
18  position: absolute;
19  top: 50%;
20  left: 50%;
21  transform: translate(-50%, -50%);
22  mix-blend-mode: overlay;
23 }
```



Modern Cityscape Text Effect

This example creates a modern text effect with a cityscape background image.

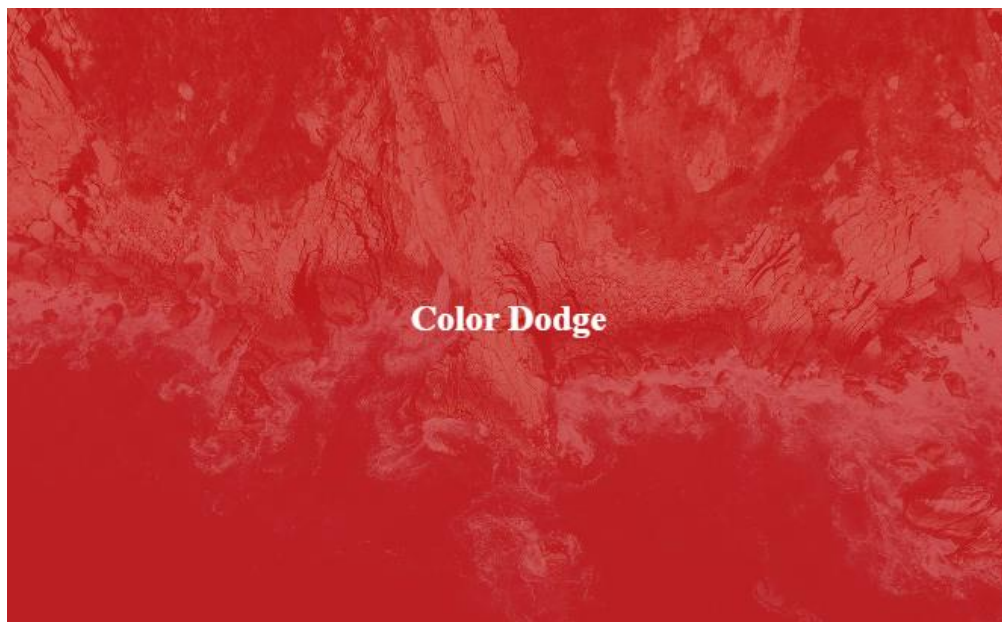
Resize the browser window to see the responsive effect.

13.4.2 Mezclar un color con la imagen de fondo con color-dodge

Este ejemplo crea un contenedor con una imagen de fondo y un `div` superpuesto con el texto “Color Dodge”. El `div` superpuesto tiene un fondo rojo con una opacidad del 70% y utiliza la propiedad `mix-blend-mode` con el valor “color-dodge” para mezclar el color rojo con la imagen de fondo, creando un efecto de mezcla de colores.



```
HTML CSS
1 * <div class="container">
2   <div class="overlay"></div>
3 * <div class="text">Color Dodge</div>
4 </div>
1 * body {
2   margin: 0;
3   padding: 0;
4   height: 100vh;
5 }
6 * .container {
7   width: 100%;
8   height: 100%;
9   background-image: url('https://www.fotos.com/wp-
content/uploads/home-historia.jpg');
10  background-size: cover;
11 }
12 * .overlay {
13   width: 100%;
14   height: 100%;
15   background-color: #ff0000;
16 * mix-blend-mode: color-dodge; /* Modo de mezcla de colores */
17 * opacity: 0.7; /* Opacidad del color-dodge */
18 }
19 * .text {
20   position: absolute;
21   top: 50%;
22   left: 50%;
23   transform: translate(-50%, -50%);
24   color: white;
25   font-size: 24px;
26   font-weight: bold;
27   text-align: center;
28 }
```





13.4.3 Otros efectos interesantes

- 1) **Destacado de elementos interactivos.** Utiliza el modo “**screen**” en botones o **enlaces** para resaltarlos sutilmente cuando el usuario pasa el cursor sobre ellos.
- 2) **Transiciones de imágenes suaves.** Combina imágenes de fondo con Blend Modes como “**Multiply**” o “**Soft Light**” para crear transiciones suaves entre diferentes secciones de tu sitio web.
- 3) **Creación de sombras y efectos de profundidad:** Aplica Blend Modes como “**Multiply**” a capas de sombra para simular efectos de profundidad y realismo en elementos de diseño.
- 4) **Mezcla de colores para fondos gradientes:** Experimenta con diferentes modos de mezcla en fondos gradientes para crear efectos de color llamativos.



14. El modo oscuro y claro con CSS

A medida que los dispositivos electrónicos forman parte integral de nuestra vida cotidiana, la experiencia del usuario se ha vuelto esencial. Una de las características más solicitadas en la interfaz de usuario es la capacidad de cambiar entre el modo oscuro y el modo claro. Veamos cómo se puede hacer con CSS utilizando la propiedad *prefers-color-scheme*.

14.1 ¿Qué es prefers-color-scheme?

prefers-color-scheme es una característica de *Media Query* introducida en CSS para **detectar** si el **usuario** tiene una **preferencia de esquema de color establecida en su sistema operativo**. Esta característica permite a los desarrolladores adaptar la estética de sus aplicaciones y sitios web al esquema de color preferido del usuario, ya sea oscuro o claro.

14.2 Cómo usar prefers-color-scheme

Para empezar, podemos usar la propiedad dentro de una consulta de medios para aplicar estilos específicos según la preferencia del usuario:

```
/* Estilos por defecto (modo claro) */
body {
    background-color: white;
    color: black;
}

/* Estilos para el modo oscuro, si así lo tiene configurado el usuario
en el sistema operativo */
@media (prefers-color-scheme: dark) {
    body {
        background-color: black;
        color: white;
    }
}
```

Con el código anterior, si un usuario ha establecido su preferencia a modo oscuro en su sistema operativo, nuestro sitio web se adaptará automáticamente para reflejar esta preferencia.



Veamos un ejemplo sencillo de una página web que puede alternar entre modos claro y oscuro utilizando ***prefers-color-scheme*** y un **botón** para permitir a los usuarios cambiar manualmente entre los dos modos usando JS.

- **HTML:**

```
<header>
  <h1>Modo Oscuro / Claro</h1>
  <button id="toggleMode">Cambiar Modo</button>
</header>
<main>
  <p>¡Bienvenido a mi sitio web! Aquí podrás cambiar entre modos
    claro y oscuro.</p>
</main>
```

- **CSS:**

```
body {
  font-family: Arial, sans-serif;
  transition: background-color 0.3s, color 0.3s; }

/* Estilos predeterminados (modo claro) */
body {
  background-color: white;
  color: black; }

header {
  padding: 20px;
  display: flex;
  justify-content: space-between;
  align-items: center; }

button {
  padding: 10px 20px;
  cursor: pointer;
  border: none;
  border-radius: 5px;
  background-color: gray;
  color: white; }

/* Estilos para modo oscuro (detectado automáticamente) */
@media (prefers-color-scheme: dark) {
  body {
    background-color: #333;
    color: white; }
}

/* Estilos para modo oscuro (cambiado manualmente) */
[data-theme='dark'] {
  background-color: #333;
  color: white; }
```



- JS:

```
const toggleButton = document.getElementById('toggleMode');

toggleButton.addEventListener('click', () => {
  if (document.body.getAttribute('data-theme') === 'dark') {
    document.body.removeAttribute('data-theme');
  } else {
    document.body.setAttribute('data-theme', 'dark');
  }
});
```



15. Scroll Snap en CSS

El **desplazamiento o scroll** es la acción de mover el contenido de una página web hacia arriba o hacia abajo para ver áreas no visibles en la pantalla. En un desplazamiento convencional, **el usuario puede deslizar el contenido de forma continua, y este se mueve en función de la dirección y la intensidad del deslizamiento.**

Sin embargo, con el **Scroll Snap** de CSS, podemos modificar este comportamiento para que **el desplazamiento se alinee con ciertos puntos de anclaje dentro del contenedor.** Por ejemplo, al desplazar hacia abajo en una página web, en lugar de permitir un movimiento fluido, **el desplazamiento puede detenerse automáticamente en ciertos puntos predefinidos, como el inicio de cada sección o bloque de contenido.**

Esto proporciona una experiencia de **navegación más estructurada y fácil de seguir para el usuario, ya que cada movimiento de desplazamiento se alinea con una sección específica de la página.**

Para implementar Scroll Snap en CSS, aplicaremos la propiedad **scroll-snap-type** al contenedor correspondiente con la siguiente **sintaxis**.

```
.contenedor-scroll {  
  scroll-snap-type: [x | y | block | inline] [mandatory | proximity]; }  
  
.seccion {  
  scroll-snap-align: [start | center | end | none]; }
```

- La propiedad **scroll-snap-type** especifica cómo se debe comportar el desplazamiento dentro del contenedor. Puede definir el tipo de desplazamiento en los ejes horizontal (**x**) o vertical (**y**), así como también en bloques (**block**) o en líneas (**inline**). Además, determina si el desplazamiento debe detenerse obligatoriamente en los puntos de anclaje especificados (**mandatory**) o si puede detenerse en el punto más cercano (**proximity**).
- Un aspecto importante a tener en cuenta es la propiedad **scroll-snap-align**, que se aplica a los elementos hijos del contenedor. Esta propiedad determina cómo se alineará cada elemento con respecto al punto de anclaje. Los valores comunes



para `scroll-snap-align` son `start`, `end`, `center`, `none`, etc. Por ejemplo, `scroll-snap-align: start`; alinea el inicio del elemento con el punto de anclaje.

Ejemplo:

Supongamos que tenemos un contenedor que contiene varias secciones y queremos aplicar Scroll Snap para que el desplazamiento se detenga en cada sección. Para ello, creamos un contenedor con la clase “contenedor-scroll” que contiene varias secciones con la clase “sección”. Aplicamos Scroll Snap al contenedor verticalmente (`scroll-snap-type: y mandatory`), lo que hace que el desplazamiento se detenga obligatoriamente en cada sección. Además, establecemos `scroll-snap-align: start` en cada sección para alinear el inicio de cada una con el punto de anclaje. Esto garantiza un desplazamiento suave y centrado entre las secciones de contenido.

HTML:

```
<div class="contenedor-scroll">
  <div class="seccion">Sección 1</div>
  <div class="seccion">Sección 2</div>
  <div class="seccion">Sección 3</div>
  <div class="seccion">Sección 4</div>
</div>
```

CSS:

```
.contenedor-scroll {
  height: 300px; /* Altura del contenedor */
  overflow-y: scroll; /* Habilitar el desplazamiento vertical */
  scroll-snap-type: y mandatory; /* Desplazamiento vertical obligatorio */
}

.seccion {
  height: 300px; /* Altura de cada sección */
  scroll-snap-align: start; /* Alinear el inicio de cada sección con el punto de anclaje */
}
```



Puedes ver su comportamiento en la siguiente url: <https://codepen.io/Eniun/pen/ExJqaMQ>

15.1 Ejemplos prácticos

A continuación, veremos algunos ejemplos de cómo podemos usar Scroll Snap para mejorar la experiencia de usuario en diferentes situaciones.

15.2 Secciones de contenido que ocupan el 100% del alto

Con Scroll Snap, podemos crear secciones de contenido que ocupen todo el alto de la pantalla y se desplacen de manera suave. Esto es útil para presentaciones de una sola página donde cada sección se alinearán perfectamente y los usuarios podrán navegar de manera intuitiva por el contenido (<https://codepen.io/Eniun/pen/oNOKvZO>)

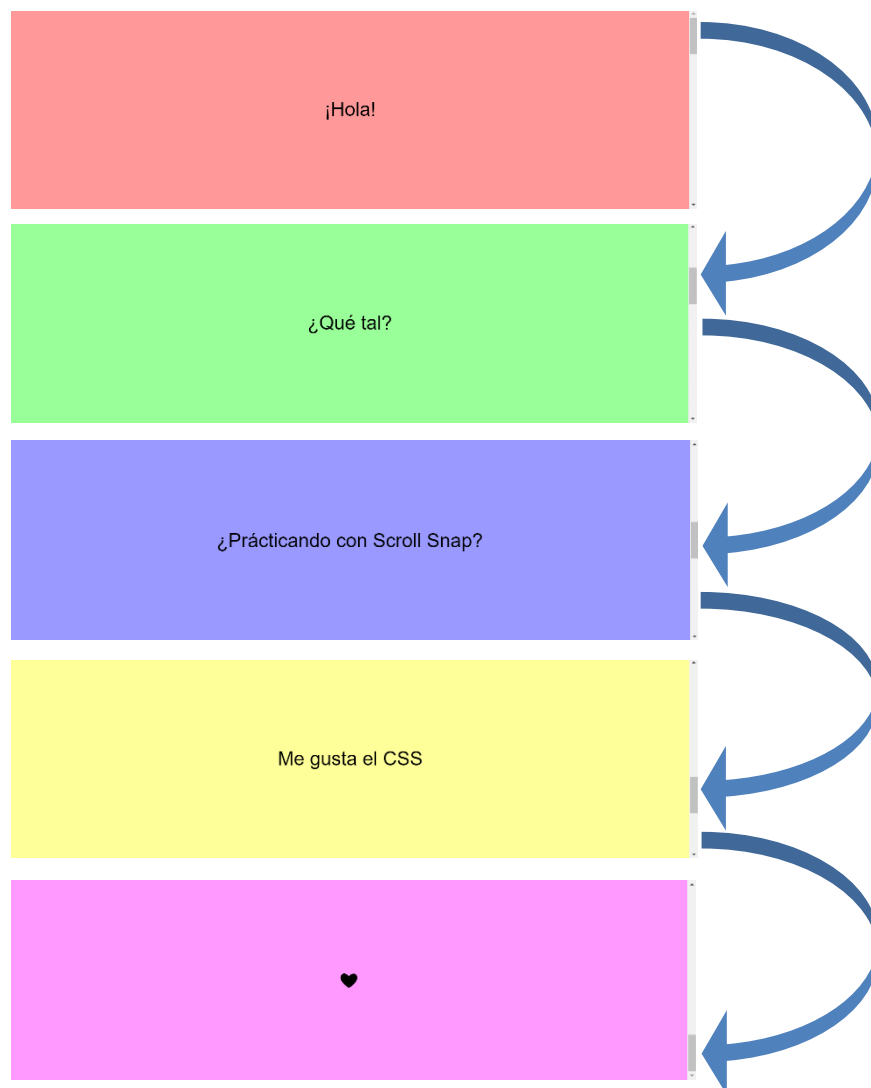
CSS

```
body, html {  
    height: 100%;  
}  
  
.scroll-container {  
    width: 100%;  
    height: 100%;  
    overflow-y: auto;  
    scroll-snap-type: y mandatory; /* Activamos el desplazamiento por defecto */  
}  
  
.scroll-item {  
    height: 100vh; /* Ocupar toda la altura de la ventana */  
    scroll-snap-align: start; /* Alineamos los elementos al inicio de la ventana de visualización */  
    display: flex;  
    justify-content: center;  
    align-items: center;  
}
```



HTML

```
<div class="scroll-container">  
  <div class="scroll-item item1">¡Hola!</div>  
  <div class="scroll-item item2">¿Qué tal?</div>  
  <div class="scroll-item item3">¿Practicando con Scroll Snap?</div>  
  <div class="scroll-item item4">Me gusta el CSS</div>  
  <div class="scroll-item item5">♥</div>  
</div>
```





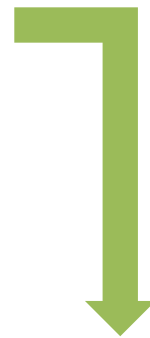
15.3 Galerías de productos en contenedores horizontales

En tiendas online, las galerías de productos pueden contar con Scroll Snap para permitir a los usuarios explorar los diferentes productos de forma horizontal. Cada producto se muestra en su propia tarjeta o *card* y el Scroll Snap asegura que cada tarjeta esté alineada perfectamente cuando se hace scroll horizontal, facilitando la comparación entre productos.

(<https://codepen.io/Eniun/pen/zYXgvYQ>)

CSS:

```
.product-gallery {  
  display: flex;  
  overflow-x: scroll;  
  scroll-snap-type: x mandatory;  
}  
  
.product-card {  
  flex: 0 0 auto;  
  width: 200px; /* Ancho tarjeta */  
  margin-right: 10px; /* Espacio entre cada tarjeta */  
  scroll-snap-align: start;  
}
```



15.4 Galería vertical con texto descriptivo

En esta ocasión, vamos a crear una galería de imágenes con texto dispuesto de forma vertical. Esto puede ser útil cuando queremos mostrar imágenes con descripciones o detalles adicionales debajo de cada imagen. **Cada imagen está alineada al inicio del contenedor**



gracias a la propiedad `scroll-snap-align`. Al hacer scroll, nos detenemos en cada elemento. Esto puede ser útil para presentaciones de productos, portafolios o cualquier otro tipo de contenido visual que queramos destacar. (<https://codepen.io/Eniun/pen/OJGKPrL>)

CSS:

```
.gallery {  
  height: 100vh;  
  overflow-y: scroll;  
  scroll-snap-type: y mandatory;  
}  
  
.image {  
  scroll-snap-align: start; /* Hace que cada imagen se alinee al inicio del  
  contenedor */  
}
```



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod





16. Scroll-behaviour CSS

Con la **propiedad CSS scroll-behavior**, podemos transformar estas transiciones y hacer que el **desplazamiento entre marcadores sea suave y animado**, sin la necesidad de recurrir a **JavaScript, JQuery** o librerías externas.

En esencia, la propiedad **scroll-behavior de CSS** nos permite agregar un efecto de scroll suave (*smooth scroll*) a nuestra página web para **mejorar la experiencia del usuario**, haciendo que la navegación sea más atractiva y fluida. De esta forma, al pulsar enlaces que llevan a un ancla (*anchor*) o marcador de una zona del documento HTML, en lugar de moverse directa e instantáneamente, que es el comportamiento por defecto del navegador, se moverá con una pequeña animación suave.

16.1 Propiedad CSS scroll-behavior: sintaxis y valores

La propiedad scroll-behavior **se aplica típicamente al elemento html** para controlar el comportamiento de desplazamiento en toda la página. Veamos su sintaxis.

```
html{  
  scroll-behavior: auto | smooth;  
}
```

- **Auto.** Este es el valor predeterminado. Indica que el desplazamiento será instantáneo, similar al comportamiento estándar del navegador.
- **Smooth.** Este valor activa el desplazamiento suave, lo que significa que el movimiento entre los diferentes elementos de la página será gradual y animado.

16.2 Diferencias entre scroll-behavior y scroll-snap

La propiedad scroll-behavior y la funcionalidad de scroll-snap son **herramientas de CSS para mejorar la experiencia de desplazamiento en las páginas web**. Aunque ambas están diseñadas para mejorar la navegación, tienen propósitos y funcionalidades diferentes. Por un lado, scroll-behavior se centra en el comportamiento general del desplazamiento entre elementos en la ventana del navegador, mientras que, por otro lado, scroll-snap se utiliza



para controlar el desplazamiento dentro de contenedores específicos, permitiendo un desplazamiento centrado en puntos de anclaje definidos.

Ambas **características son complementarias** y pueden mejorar la experiencia de navegación en una página web al proporcionar un desplazamiento más suave, fluido y controlado.

16.3 Ejemplos prácticos de scroll-behavior

Veamos algunos ejemplos prácticos que puedes necesitar en la creación de tus páginas web para conseguir efectos que mejoren la **usabilidad**.

16.3.1 Menú de navegación con enlaces a secciones con scroll suave

La propiedad scroll-behavior es muy utilizada en **páginas de aterrizaje (landing pages)** compuestas por un menú principal que enlaza a diversos marcadores o anclas dispuestos en distintos contenedores.

HTML :

```
<nav>
  <ul>
    <li><a href="#section1">Sección 1</a></li>
    <li><a href="#section2">Sección 2</a></li>
    <li><a href="#section3">Sección 3</a></li>
    <li><a href="#section4">Sección 4</a></li>
  </ul>
</nav>

<section id="section1">Sección 1</section>
<section id="section2">Sección 2</section>
<section id="section3">Sección 3</section>
<section id="section4">Sección 4</section>
```



```
html{
  scroll-behavior: smooth; }

body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0; }

nav {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background-color: #333;
  padding: 10px 0;
  z-index: 999; }

nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
  text-align: center; }

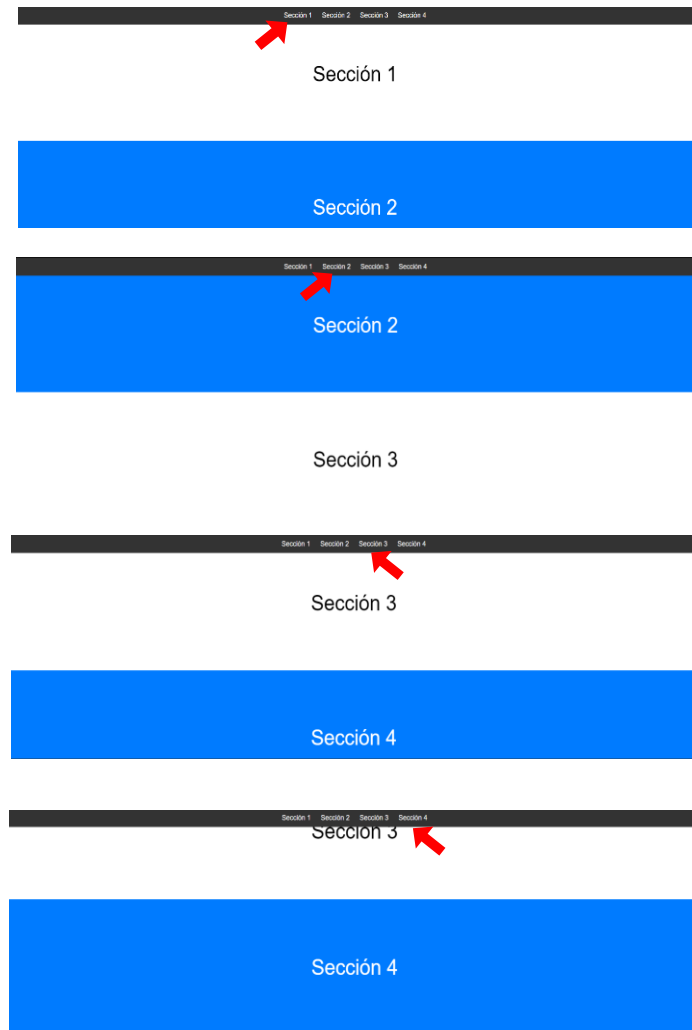
nav ul li {
  display: inline-block; }

nav ul li a {
  color: #fff;
  text-decoration: none;
  font-size: 16px;
  padding: 10px;
  transition: background-color 0.3s; }

nav ul li a:hover {
  background-color: #555; }

section {
  height: 300px;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 3rem;
  color: black; }

section:nth-child(odd) {
  background-color: #007bff;
  color: white; }
```

16.3.2 Botón para volver a la parte superior de la página con scroll suave

Este ejemplo muestra un botón al final de la página que, al hacer clic en él, desplaza la página suavemente hasta la parte superior. Es una forma de proporcionar a los usuarios una manera rápida de volver al inicio de la página. De la misma forma, se podría hacer el típico botón fijo lateral con una flecha hacia arriba que nos permita volver hacia arriba en cualquier momento.

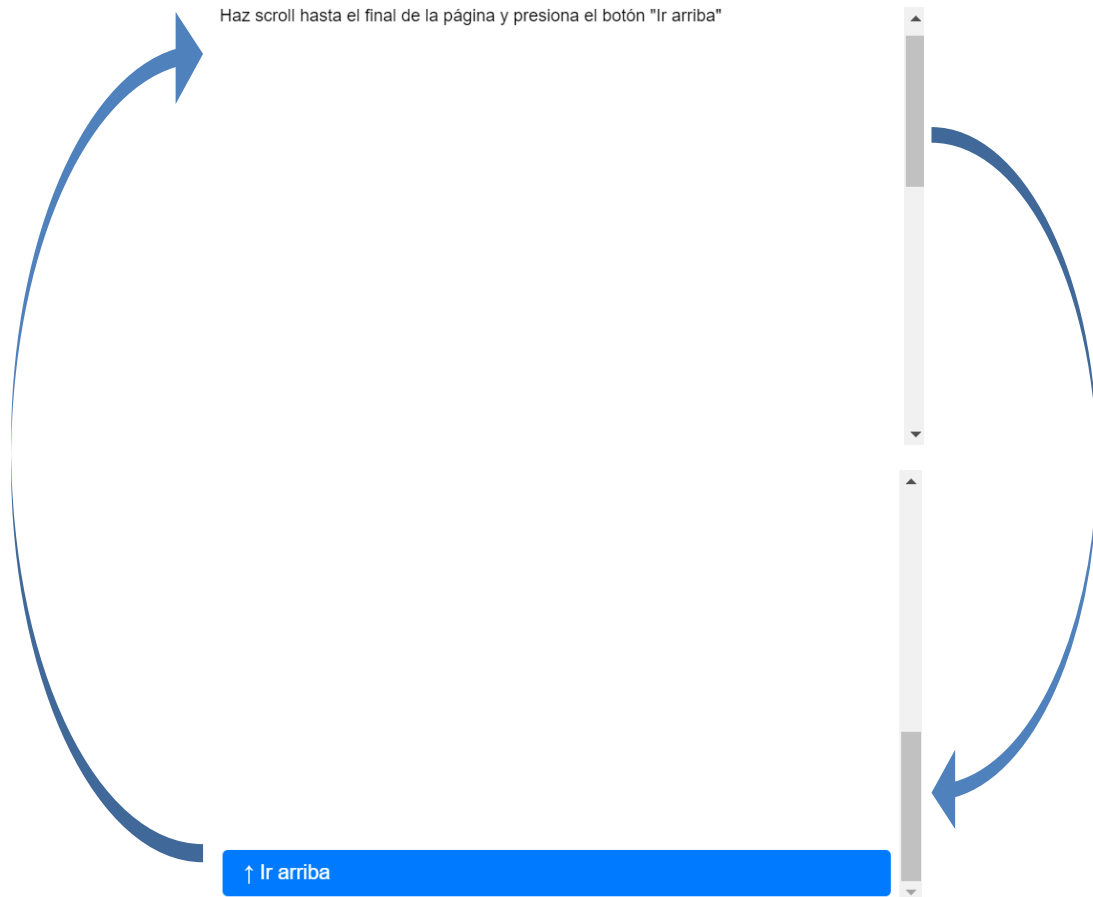
HTML:

```
<div class="container" id="scrollBtn">  
<p>Haz scroll hasta el final de la página y presiona el botón "Ir  
arriba"</p>  
</div>  
<a href="#scrollBtn">↑ Ir arriba</a>
```



CSS:

```
html{  
    scroll-behavior: smooth;  
}  
  
body {  
    font-family: Arial, sans-serif;  
}  
  
.container {  
    height: 1000px;  
}  
  
a {  
    display: flex;  
    padding: 10px 20px;  
    font-size: 1.2rem;  
    background-color: #007bff;  
    color: #fff;  
    border: none;  
    border-radius: 5px;  
    cursor: pointer;  
    text-decoration: none;  
    transition: background-color 0.3s ease;  
}  
  
a:hover {  
    background-color: #0056b3;  
}
```



16.3.3 Aside de contenidos con scroll suave

Este ejemplo muestra un diseño con un aside de contenidos a la izquierda y el contenido principal a la derecha. Los enlaces en el aside te llevan a las diferentes secciones del contenido principal, y el scroll es suave gracias a la propiedad scroll-behavior de CSS. (

HTML:

```
<div class="container">
  <aside>
    <ul>
      <li><a href="#section1">Sección 1</a></li>
      <li><a href="#section2">Sección 2</a></li>
      <li><a href="#section3">Sección 3</a></li>
      <li><a href="#section4">Sección 4</a></li>
    </ul>
  </aside>
```



```
<main>
  <section id="section1">
    <h2>Sección 1</h2>
    <p>...</p>
  </section>
  <section id="section2">
    <h2>Sección 2</h2>
    <p>...</p>
  </section>
  <section id="section3">
    <h2>Sección 3</h2>
    <p>...</p>
  </section>
  <section id="section4">
    <h2>Sección 4</h2>
    <p>...</p>
  </section>
</main>
</div>
```

CSS:

```
html{
  scroll-behavior: smooth; }

body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0; }

.container {
  display: flex;
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px; }
```



```
aside {
  flex: 0 0 200px;
  background-color: #f0f0f0;
  padding: 20px;
}
main {
  flex: 1;
  padding: 20px;
}
ul {
  list-style-type: none;
  padding: 0;
}
li {
  margin-bottom: 10px;
}
a {
  text-decoration: none;
  color: #007bff;
}
a:hover {
  text-decoration: underline;
}
section {
  margin-bottom: 40px;
}
section:last-child {
  margin-bottom: 0;
}
```





17. Preprocesadores CSS

Los preprocesadores de CSS son lenguajes de programación **que amplían las capacidades predeterminadas de CSS**. Nos permiten utilizar lógica en nuestro código CSS, como variables, anidamiento, herencia, mixins, funciones y operaciones matemáticas. Los preprocesadores de CSS facilitan la automatización de tareas repetitivas, reducen el número de errores y la saturación del código, crean fragmentos de código reutilizables y garantizan la compatibilidad con versiones anteriores.

Mediante los preprocesadores CSS **se desarrollan estilos más mantenibles y con una sintaxis más rica**. Los ficheros de los preprocesadores son parecidos a los de CSS y **deben compilarse** para traducirse a hojas de estilo CSS reconocibles por los diferentes navegadores.

La **compilación** se realiza generalmente mediante **herramientas de línea** de comandos o con herramientas de automatización de tareas (como Gulp o Grunt).

Por tanto, **cada preprocesador de CSS tiene su propio ecosistema** (herramientas, frameworks, bibliotecas), **y su propia sintaxis** que compilan en CSS normal para que los navegadores puedan renderizarlo en el lado del cliente.

Los preprocesadores más utilizados son **Less** y **Sass**,

17.1 Preprocesadores Less y Sass

Una de las principales ventajas de utilizar preprocesadores como Less y Sass es la capacidad de **definir y reutilizar variables**. Esto permite establecer valores que se utilizan repetidamente en todo el código, lo que facilita enormemente la actualización y mantenimiento del diseño. Por ejemplo, si se decide cambiar el color primario de un sitio web, simplemente se modifica la variable correspondiente y esta actualización se reflejará automáticamente en todos los lugares donde se utiliza esa variable. **Actualmente, CSS ya cuenta con variables personalizadas**, como veremos en el siguiente apartado.

Además de las variables, los preprocesadores CSS también introducen la noción de **anidamiento**, lo que permite escribir estilos de manera más jerárquica y estructurada. Esto significa que los selectores secundarios pueden anidarse dentro de selectores primarios, lo que resulta en un código más limpio y legible. Por ejemplo, en lugar de repetir



el selector completo para cada regla CSS, como se hace en CSS tradicional, con los preprocesadores se pueden anidar las reglas dentro de los selectores correspondientes, lo que hace que el código sea más fácil de entender y mantener.

Otra característica de los preprocesadores CSS es la capacidad de crear **mixins**, que son fragmentos de código reutilizables que se pueden incluir en otros estilos. Esto facilita la aplicación de estilos comunes a múltiples elementos sin tener que repetir el mismo código una y otra vez. Por **ejemplo**, se puede definir un mixin para aplicar un efecto de transformación a un elemento y luego simplemente incluir ese mixin en los estilos de cualquier elemento que necesite ese efecto. Veamos algunos ejemplos de código.

Ejemplos Sass:

Variables:

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
    font: $font-stack;
    color: $primary-color;
}
```

Nesting:

```
nav {
    ul {
        margin: 0;
        padding: 0;
        list-style: none;
    }
    li {
        display: inline-block;
    }
}
```




Operators:

```
article {  
    float: left;  
    width: 600px / 960px * 100%;  
}  
aside {  
    float: right;  
    width: 300px / 960px * 100%;  
}
```

Mixins:

```
@mixin transform($property) {  
    -webkit-transform: $property;  
    -ms-transform: $property;  
    transform: $property;  
}  
.box {  
    @include transform(rotate(30deg));  
}
```

Conoce más acerca de los **preprocesadores Sass y Less** en los siguientes enlaces:

- [Aprende Sass](#)
- [Aprende Less](#)

18. CSS Nesting Nativo (Anidamiento CSS Nativo)

Tradicionalmente, el código CSS se ha trabajado siempre mediante selectores CSS. Sin embargo, la idea detrás del concepto de **CSS Nesting** (código CSS anidado) es tener **fragmentos o bloques de código uno dentro de otros**, haciendo que estos selectores sean mucho más intuitivos para el programador y, por lo tanto, mucho más fáciles de mantener. Veamos un **ejemplo** muy sencillo.

Con CSS *tradicional*:

```
.container {  
  width: 800px;  
  height: 300px;  
  background: grey;  
}  
  
.container .item {  
  height: 150px;  
  background: orangered;  
}
```



Con CSS *anidado*:

```
.container {  
  width: 800px;  
  height: 300px;  
  background: grey;  
}  
  
.item {  
  height: 150px;  
  background: orangered;  
}
```

Utilizando **CSS Nesting**, sin embargo, conseguimos que sólo mirando el código se haga evidente cuando un elemento está dentro de otro, aislando código, haciendo mucho más **fácil de mantener y modularizar** nuestros estilos

A lo largo del tiempo, el CSS Nesting se ha implementado con diferentes sintaxis (Sass, LESS, stylus, PostCSS, etc...), pero todas ellas se podían utilizar sólo mediante herramientas que convertían el código anidado en código estándar de CSS sin anidado, ya que no estaba soportado en navegadores.

Sin embargo, finalmente, se ha implementado **de forma nativa** (desde 2023), en la mayoría de navegadores en **dos formatos**:

- 1) **Sintaxis avanzada**: mediante el uso del **símbolo &** para indicar el elemento padre. Esto es, hace que los **selectores hijo anidados sean relativos al elemento padre inmediato**.
- 2) **Sintaxis simple o relaxed**: se omite el uso del símbolo **&**. En este caso, en código nativo CSS, equivaldría a selectores descendientes.



18.1 Sintaxis relaxed

La sintaxis más sencilla que podemos utilizar es la que mostramos en el ejemplo anterior. Simplemente, **dentro de un elemento, podemos volver a mencionar otro**. Sin embargo, aunque es más sencilla y sirve para casos simples, esta sintaxis puede tener ciertas **limitaciones**.

Con CSS *tradicional*:

```
.container {  
  width: 800px;  
  height: 300px;  
  background: grey;  
}  
  
.container .item {  
  height: 150px;  
  background: orangered;  
}
```



Con CSS *anidado (sintaxis relaxed)*:

```
.container {  
  width: 800px;  
  height: 300px;  
  background: grey;  
  
  .item {  
    height: 150px;  
    background: orangered;  
  }  
}
```

El anidamiento de .item corresponde al selector .container .item. Sin embargo, hay varios casos donde este anidamiento no funcionaría y tendríamos que utilizar el selector &.



Safari, en el motor WebKit ha establecido que el [máximo de niveles de anidamiento sea 128](#). Sin embargo, aunque no tenemos un valor estricto, deberíamos intentar **mantenernos como máximo en 4-5 niveles** de anidamiento aproximadamente, y sólo superarlos si está bien justificado.



18.2 Sintaxis avanzada

Dentro del CSS Nesting tenemos el **operador &**, que nos permite hacer **referencia al selector inmediatamente padre** dentro del anidamiento. Esto puede resultarnos muy útil en algunos casos diferentes al anterior.

Observa el siguiente **ejemplo**, donde en lugar de la clase `.item` seleccionamos cualquier elemento `div` que esté dentro de `.container`.

```
.container {  
  background: grey;  
  
  & div {  
    background: indigo;  
  }  
}
```

El ejemplo anterior, **sin el selector & es imposible de realizar con CSS Nesting**, puesto que la sintaxis simple no permite colocar elementos directos que no sean clases, id, combinadores, etc.



Otra situación problemática podría ser el caso en el que queramos darle estilo a la clase `.container` cuando tenemos el ratón encima de ella. Observa el siguiente fragmento de código donde conseguimos un selector equivalente a `.container:hover`:

```
.container {  
  background: grey;  
  
  &:hover {  
    background: indigo;  
  }  
}
```

Si omitimos el selector `&` en este ejemplo anterior, el código CSS será válido, pero no equivalente.

Omitiendo el selector `&` estaríamos aplicando estilos `:hover` a los elementos dentro de `.container` en lugar de al propio `.container`.





18.3 Anidamiento sobre el padre

En los casos anteriores siempre hemos colocado el selector **&** al principio. Sin embargo, puede darse el caso que queramos **anidar de una forma diferente**.

En el siguiente **ejemplo**, el **selector equivalente** sería **.item .container**, es decir, elementos **.item** que contengan al padre:

```
.container {  
  width: 800px;  
  height: 300px;  
  background: grey;  
  
  .item & {  
    background: green;  
    height: 100px;  
  }  
}
```

Además, también nos permite hacer anidaciones menos directas pero que pueden ser muy interesantes por su relación con el elemento contenedor. Observa los siguientes **ejemplos**:

```
.container {  
  width: 800px;  
  height: 300px;  
  background: grey;  
  
  .item & .item & { } /* Equivalente a → .item .container .item .container */  
  :is(.page, .menu) & { } /* Equivalente a → .page .container, .menu .container */  
  :not(&) { } /* Equivalente a → :not(.container) */  
}
```

NOTA. Los elementos **:is()**, **:not()**, **:where()**, y **:has()** son **pseudoclases nuevas**.

Podrás obtener más información en los siguientes enlaces:

:is() y **:where()**: <https://lenguajecss.com/css/selectores/combinador-is-where/>

:not(): <https://lenguajecss.com/css/selectores/combinador-not/>

:has(): <https://lenguajecss.com/css/selectores/combinador-has/>



18.4 Anidamiento de reglas

Es posible anidar reglas CSS en el interior de bloques de código CSS donde se aplica CSS Nesting. Concretamente, es posible hacerlo si se trata de reglas `@media`, `@supports`, `@layer`, `@scope` o `@container`. De esta forma, podemos organizar, por ejemplo, código responsive de un fragmento de código de forma muy compacta:

```
.container {  
  height: 200px;  
  background: grey;  
  
  @media (orientation: landscape) {  
    height: 100vh;  
  }  
}
```



19. Anexo I - SaSS

SaSS es un preprocesador CSS que traduce un código de hojas de estilo **NO estándar** a un código CSS estándar legible por la mayoría de los navegadores.

SaSS es el preprocesador más utilizado por los desarrolladores web de frontend hoy en día.

La principal utilidad de SaSS es la de hacer **más simple la escritura del código CSS**, además de brindar diversas utilidades que a día de hoy CSS no puede ofrecer.

Entre otras posibilidades, SaSS permite:

- Usar **variables**.
- Realizar **cálculos** con operadores.
- **Anidar** selectores para escribir menos código.
- Usar **funciones** para no tener que repetir el código similar.
- **Separar** el **código** del proyecto en varios ficheros para un mantenimiento más sencillo.

Tutoriales de SaSS:

- <https://www.w3schools.com/sass/>
- <https://sass-lang.com/documentation/>
- <https://sass-lang.com/guide/>

19.1 Instalación de SaSS

La instalación de SaSS se puede hacer mediante aplicaciones (son de pago) o mediante línea de comandos: <https://sass-lang.com/install/>

En este apartado vamos a centrarnos en la instalación por línea de comandos. Simplemente lo que hay que hacer es descargar el paquete correspondiente al sistema operativo que utilizamos desde [GitHub](#) y añadirla a nuestro [PATH](#).

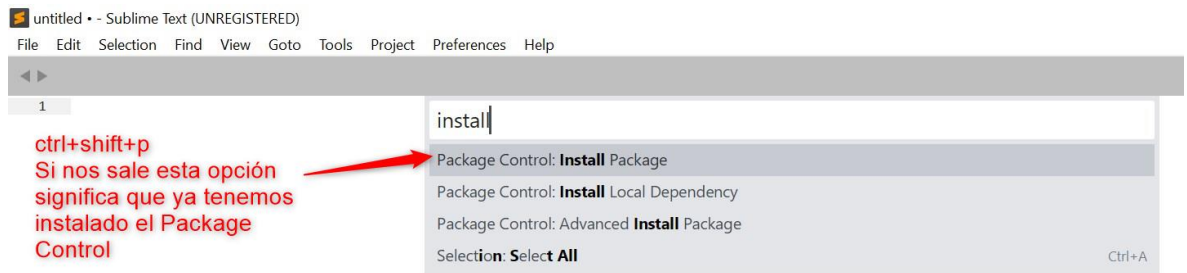
- Si usamos Visual Studio, debemos instalar un plugin que nos permite trabajar con archivos *.sass y *.scss y procesarlos en tiempo real.

<https://marketplace.visualstudio.com/items?itemName=ritwickdey.live-sass>

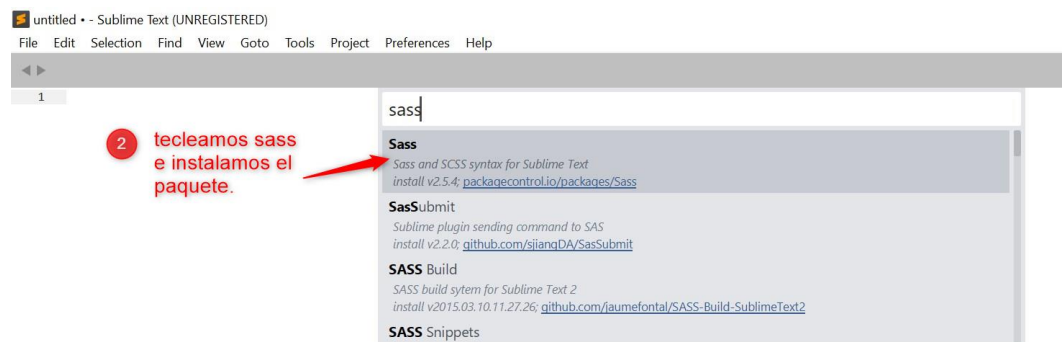
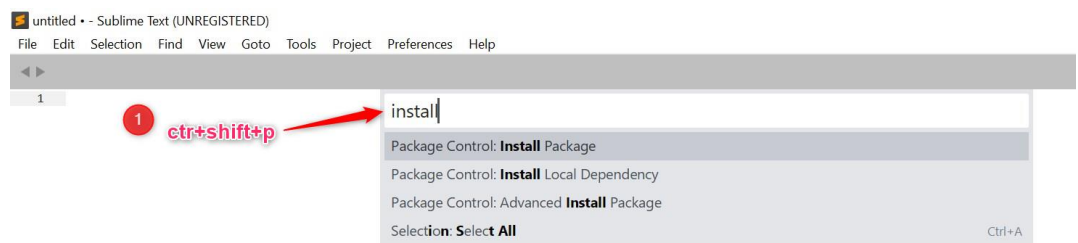


- Si usamos SublimeText debemos instalar lo siguiente:

1) Package Control (<https://packagecontrol.io/installation>)



2) Paquete de SaSS del Package Control (<https://packagecontrol.io/packages/Sass>)



19.2 Formatos

SASS dispone de dos formatos diferentes para la sintaxis, lo que hace se traduce en dos extensiones de fichero diferentes: **.sass** y **.scss**. De ambas extensiones hay que elegir una.

El primero en salir fue **.sass** y se caracteriza, al igual que Stylus y [coffeescript](#) para el lenguaje de programación Javascript, en no hacer uso de llaves ni punto y coma final, en busca de la simplicidad y eliminación de ruido.



```
body
background: #000
font-size: 62.5%
```

Los **.scss** salieron con la versión 3 de preprocesador y permiten utilizar llaves e incorporar código de CSS clásico.

```
body {
background: #000;
font-size: 62.5%;
}
```

Tanto la sintaxis de **Sass** como la de **.scss** no puede ser interpretada directamente por los navegadores, por lo se debe compilar para traducir nuestro archivo SASS en un clásico fichero CSS.

Por ejemplo, si tenemos el siguiente código CSS:

```
body {
  background: #f2f2f2;
}

.articulo {
  background: #fff; }

.articulo .titulo {
  color: #000; }

.articulo .fecha {
  // estilos }

.articulo p a {
  text-decoration: none; }
```

Y lo queremos escribir en Sass, este tendría el siguiente aspecto:

 **.sass:**

```
// Esta syntax es mas antigua pero en vez de { } utiliza tabulaciones.

body
  background: #f2f2f2

.articulo
  background: #fff
  .titulo
    color: #000
  .fecha
    // estilos
  p
    a
      text-decoration: none
```



.SCSS:

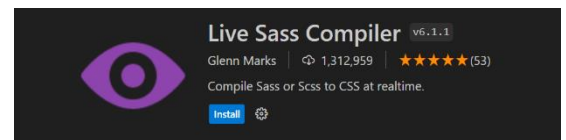
```
// Esta syntax es mas nueva pero utiliza { } y ;

body {
  background: #f2f2f2;
}

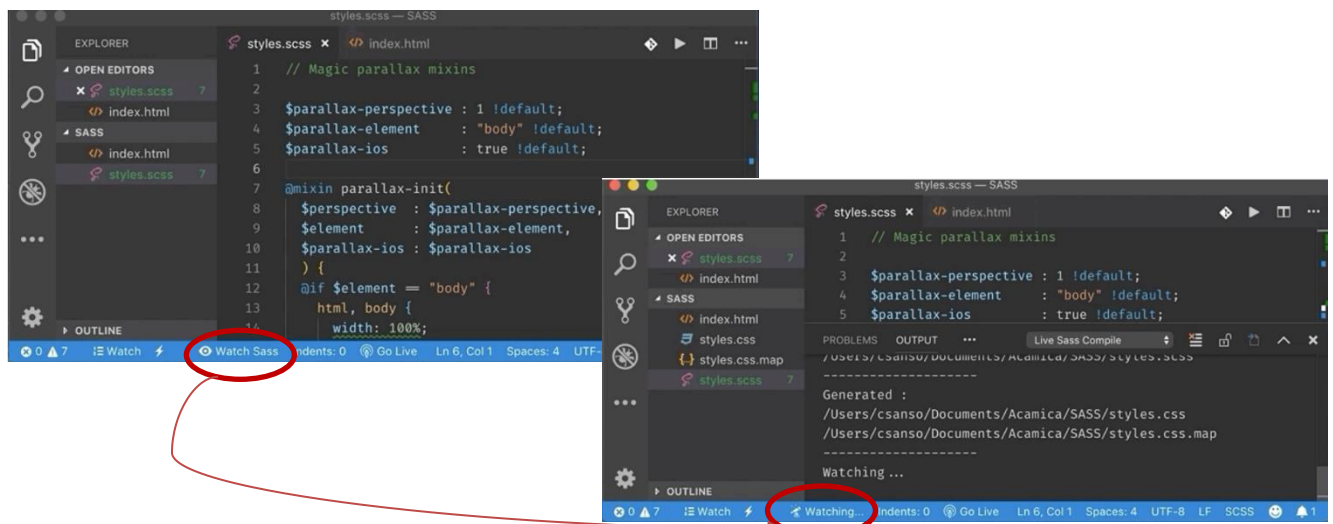
.articulo {
  background: #fff;
  .titulo {
    color: #000;
  }
  .fecha {
    // estilos
  }
  p {
    a {
      text-decoration: none;
    }
  }
}
```

19.3 Cómo compilar nuestro código

La estrategia más simple para compilar nuestro código sass o scss, es descargar la extensión '**Live Sass Compiler**' de *Visual Studio Code* que con un simple botón nos va a generar nuestro código CSS.

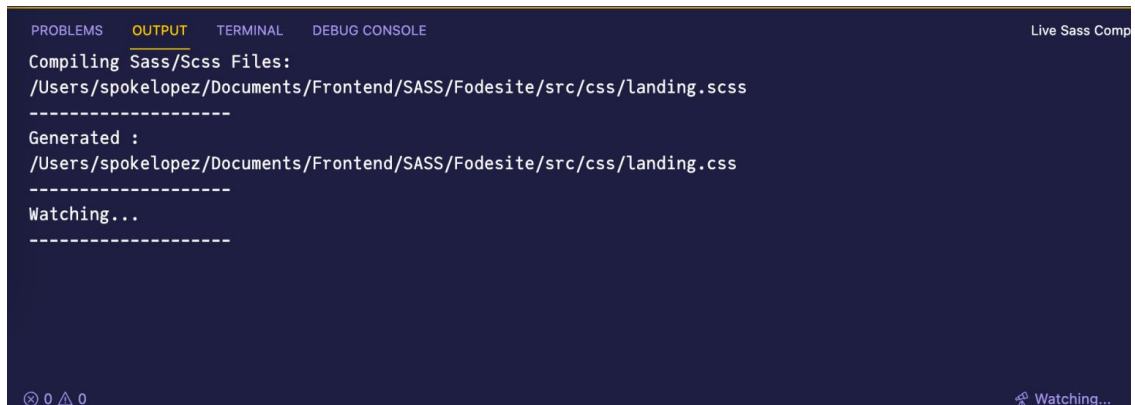


Una vez descargado y recargado *VSCode*, se puede observar que en la parte inferior aparece un nuevo botón que dice '**Watch Sass**'. Haciendo click en él, podemos ver como automáticamente nos creará dos archivos nuevos. Uno con la extensión de **.css**, que es el que vamos a usar para dar estilos, y otro es **.css.map**.





El **output** en Visual Studio Code nos mostrará siempre el status de la compilación de nuestro documento.



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE Live Sass Comp
Compiling Sass/Scss Files:
/Users/spokelopez/Documents/Frontend/SASS/Fodesite/src/css/landing.scss
-----
Generated :
/Users/spokelopez/Documents/Frontend/SASS/Fodesite/src/css/landing.css
-----
Watching...
-----
0 0 0 Watching...
```

Existen más formas para compilar nuestro código sass. Algunos de ellos son:

- 1) Utilizando **consola**. Es la forma más liviana, simple, escalable y controlada (<https://sass-lang.com/guide/#preprocessing>).
- 2) **Programas de terceros** como Prepros (de pago), Koala y Scout-app.
 - <http://koala-app.com/>
 - <https://scout-app.io/>
 - <https://prepros.io/help/sass>
- 3) **Compiladores online** como [SASSmeister](#)